TEACHME-LAKEHOUSE · TRAINING PROGRAMME

Module 0 — Basics: The Ecosystem, Decoded

00

What each word means, which service does it, who can read it, who can write it, and how data actually gets there

23 HOURS | 34 LESSONS | NO PREREQUISITES

AUTHOR
Surender Sara

COMPANY
Northbay Solutions

ISSUED
August 2026

MODULE 0 · LESSON 01

Four Words For Where Data Lives

Warehouse, lake, lakehouse, mesh. Vendors blur them deliberately. Four questions tell them apart for good.

1 · WHAT IT IS

They are four answers to the same question, not four stages of maturity

You can run more than one at a time. Most companies do. The words only become useful once you know what separates them.

2 · HOW TO TELL THEM APART

ASK THESE FOUR QUESTIONS, IN THIS ORDER

SCHEMA — `on write / on read`
Who decides the shape, and when — before the data lands, or when someone reads it?

ACCESS — `who is allowed`
Who can read it, and at what grain — whole tables, or single columns and rows?

COST — `per hour / per TB scanned`
What does one query cost, and who pays — the cluster owner, or the person asking?

OWNERSHIP — `central / federated`
Who owns it — one central data team, or the domain that produced the data?

```
answer all four and the word picks itself — you never argue about the label again
```

3 · THE SHORT ANSWER

Warehouse — modelled, trusted, costly
Lake — cheap, raw, needs governing
Lakehouse — one store, both behaviours
Mesh — an ownership model, not a store

THREE ARE PLACES · ONE IS A RULE

4 · IN PRACTICE

Tamimi and Apparel Group are both lakehouses: S3 + Iceberg + Redshift.
Neither is a mesh, and neither needs to be.

Lesson 04 is the one that matters

IN PLAIN ENGLISH

A shop floor, a garage, a garage someone fitted shelves into — and a rule about who holds the keys to each room.

MODULE 0 · LESSON 01

L01 · Four Words For Where Data Lives

Slide: `_render/L01-four-words.html`

What it is

You are going to hear four words this week — **data warehouse**, **data lake**, **lakehouse**, **data mesh** — and you will hear them used as if they were interchangeable, or as if they were stages you graduate through. They are neither.

They are four different answers to the same question: *where does the organisation's data live, and on what terms?* Most companies run more than one of them at once, deliberately.

The reason the words feel slippery is that vendors blur them. Every product is marketed as whichever word is currently fashionable. So instead of memorising definitions, learn the four questions that actually separate them — then you can classify any system, including ones that have not been invented yet.

The four questions

1. Schema — who decides the shape, and when?

- **Schema on write** — the shape is fixed before data is allowed in. Anything that does not fit is rejected at the door.
- **Schema on read** — data lands as it arrives; meaning is applied by whoever queries it later.

This one question drives most of the others. Schema-on-write buys you trust and costs you flexibility. Schema-on-read is the reverse.

2. Access — who is allowed to read it, and at what grain?

Not "is there security" — everything has security. The real question is the *granularity*: can you grant one team access to one column of one table, or is the smallest unit "the whole database"? A system that can only grant whole databases will eventually force people to make copies, and copies are where governance goes to die.

3. Cost — what does one query cost, and who pays?

Two very different billing models: - **Per hour of compute** — you rent a cluster; queries are then "free". Someone runs an accidental cross join and nobody notices. - **Per terabyte scanned** — each query has a price. A badly written query is *visibly* expensive, which changes behaviour.

Neither is better. But they produce completely different engineering cultures, and you should know which one you are in.

4. Ownership — who owns the data?

- **Central** — one data team owns every pipeline and every table.
- **Federated** — the domain that produces the data owns it, publishes it, and is on call for it.

This is an organisational question wearing a technical costume, which is exactly why Lesson 05 exists.

The short answer

WORD	ONE SENTENCE
Warehouse	Modelled, trusted, expensive — schema on write
Lake	Cheap, raw, needs governing — schema on read
Lakehouse	One store, both behaviours — a table format bridges them
Mesh	An ownership model, not a place to put data

Three of them are places. One of them is a rule about who holds the keys. Confusing that is the single most common mistake in this whole vocabulary.

L01 · Four Words For Where Data Lives

In practice

Both the Tamimi platform and the Apparel Group platform being designed are **lakehouses**: S3 for storage, Apache Iceberg for table behaviour, Redshift for the modelled reporting layer.

Neither is a mesh. One platform team owns everything end to end. That is the right call at this size, and Lesson 05 explains why.

Checklist

- ☐ I can state the four separating questions without looking
- ☐ I can classify a system I am shown by answering them
- ☐ I know which of the four words describes ownership rather than storage
- ☐ I can name what our platform is, and why

You've got it when you can…

…be shown an unfamiliar architecture diagram and say, in one sentence each, where its schema is decided, how finely access can be granted, who pays per query, and who owns the data — and then name which of the four words applies, without anyone having told you.

MODULE 0 · LESSON 02

The Data Warehouse

Schema on write. You fix the shape before the data arrives — which is why two people asking the same question get the same number.

1 · WHAT IT IS

A warehouse trades flexibility for trust

You model first and load second. That is the whole bargain — and it is why the finance team believes the number on the dashboard.

AWS: Amazon Redshift

2 · HOW IT WORKS

FOUR THINGS MAKE IT FAST, AND THEY ALL COST YOU UP FRONT

COLUMNAR — `not row-by-row`

Each column is stored together, so a query touching 3 of 200 columns reads 1.5% of the bytes.

MPP — `leader + compute nodes`

The table is split across slices and the same plan runs on every slice at once.

SCHEMA ON WRITE — `rejected at the door`

Types, keys and grain are fixed at load time. Bad rows fail on arrival, not six months later.

RMS — `Redshift Managed Storage`

Storage sits on S3 and scales to petabytes, so compute and storage scale independently.

```
SELECT is fast because you paid the cost at load time, not at query time
```

3 · WHEN TO USE IT

Repeatable business reporting
Numbers that have to reconcile
Joins across many dimensions

NOT FOR

Raw, unmodelled or unknown-shape data

4 · IN PRACTICE

Redshift Serverless holds the gold layer.
Bronze and silver stay in Iceberg on S3.
Power BI reads Redshift, never the lake.

gold = modelled = trusted

IN PLAIN ENGLISH

The shop floor before opening: everything unpacked, priced and on the right shelf. You pay staff up front so customers find things instantly.

L02 · The Data Warehouse

Slide: `_render/L02-data-warehouse.html`

What it is

A data warehouse is a database designed for **analytical** questions rather than transactional ones, where the shape of the data is decided *before* it is loaded.

That trade — flexibility for trust — is the entire point. When the finance team asks "what were sales last quarter?" twice, six weeks apart, they get the same number. That is not an accident; it is what you bought by modelling first.

On AWS: Amazon Redshift, either provisioned or Serverless, storing into **Redshift Managed Storage (RMS)** — a storage tier that sits on S3, scales to petabytes, and lets compute and storage scale independently of each other.

How it works

Columnar storage

An application database stores a row together, because applications fetch whole rows. A warehouse stores each **column** together, because analytical queries touch a few columns of very many rows.

A query reading 3 columns from a 200-column fact table reads roughly 1.5% of the bytes. That single design decision is most of the performance difference.

MPP — massively parallel processing

The table is split across **slices**, and the same query plan runs on every slice simultaneously. Adding compute adds parallelism. This is why a warehouse can scan a billion rows in seconds and an application database cannot.

Schema on write

Types, keys and grain are fixed at load time. Bad rows fail on arrival, where someone is watching, rather than six months later inside a board report.

RMS — separating storage from compute

Older warehouses coupled the two: more storage meant more nodes whether you needed the CPU or not. RMS decouples them. You size compute for your query load and let storage grow on its own.

When to use it

Use a warehouse for: - Repeatable business reporting where consistency matters more than flexibility - Numbers that have to reconcile against each other and against source systems - Queries that join across many dimensions

Do not use it for: - Raw, unmodelled or unknown-shape data — that belongs in the lake (Lesson 03) - Machine-learning feature engineering on unstructured input - Anything you have not yet decided the meaning of

The thing that surprises SQL developers

`SELECT` is fast **because you paid the cost at load time**. There is no magic. Everything that makes reads fast — sorting, distribution, compression, modelling — is work done during the load. A warehouse where loading is cheap and querying is slow has been built backwards.

In practice

On our platforms: - **Redshift Serverless holds the gold layer only** — modelled facts and dimensions. - **Bronze and silver stay in Iceberg on S3**, because they are not yet trustworthy enough to be warehouse tables and re-reading them is cheap. - **Power BI reads Redshift, never the lake.** The warehouse is the trust boundary.

L02 · The Data Warehouse

The rule to take away: *gold = modelled = trusted.* If something has not been modelled, it does not belong in the warehouse yet.

Checklist

- ☐ I can explain columnar storage to someone who only knows row storage
- ☐ I can explain why MPP makes big scans fast
- ☐ I can say what schema-on-write buys and what it costs
- ☐ I know what RMS is and why separating storage from compute matters
- ☐ I know which layer of our platform lives in Redshift, and which does not

You've got it when you can…

…explain to an application developer why the database pattern they already know — normalise, index, query — produces a *slow* warehouse, and what you do instead.

MODULE 0 · LESSON 03

The Data Lake

Schema on read. Land everything cheaply now and decide what it means later — then govern it, or it becomes a swamp.

1 · WHAT IT IS

A lake trades trust for optionality

Storage is cheap enough that keeping everything is finally affordable. What you buy is the right to ask a question you have not thought of yet.

AWS: S3 + Glue + Athena

2 · HOW IT WORKS

FOUR PIECES — AND THE FOURTH IS THE ONE PEOPLE SKIP

OBJECT STORAGE — Amazon S3
S3 holds files at cents per GB per month, with eleven nines of durability and no capacity to plan.

SCHEMA ON READ — nothing validates on arrival
The file lands as it is. Meaning is applied by whoever queries it — which is freedom, and a risk.

CATALOG — AWS Glue Data Catalog
What turns a folder of files into a named table with columns that people can actually find.

GOVERNANCE — AWS Lake Formation
Decides who sees which table — and which columns inside it. Skip this and you have a swamp.

```
no catalog + no owner = a swamp: the data is all there and nobody can use it
```

3 · WHEN TO USE IT

Raw data you must retain for years
Schemas you do not control — SaaS, logs
Data science on unmodelled data

NOT FOR

Numbers the CFO has to sign off

4 · IN PRACTICE

raw/ and bronze/ prefixes on S3.
Every file lands immutable, exactly once.
Nothing is ever edited in place.

cheap to keep · cheap to re-read

IN PLAIN ENGLISH

The garage: everything goes in as-is and it costs almost nothing. Without labels on the boxes you will never find any of it again.

MODULE 0 · LESSON 03

L03 · The Data Lake

Slide: `_render/L03-data-lake.html`

What it is

A data lake is cheap storage that accepts data in whatever shape it arrives, and defers the decision about what it means until someone queries it. That is **schema on read**.

What you are actually buying is *optionality*: the right to ask, in two years, a question nobody has thought of yet — because you still have the raw data to answer it with.

On AWS: S3 + Glue Data Catalog + Athena + Lake Formation. All four, not just the first one. A lake that is only S3 is a folder.

How it works

Object storage

S3 holds files at cents per GB per month, with eleven nines of durability and no capacity to plan or provision. "Keep everything" stopped being reckless when storage got this cheap.

Schema on read

Nothing validates the file when it lands. A column that changed type upstream lands happily and breaks a query three weeks later. That is the cost of the flexibility, and it is why Lesson 24 is about landing conventions.

The catalog

The **Glue Data Catalog** is what turns a folder of files into a *named table with columns* that a person can discover and an engine can query. Without it, the only way to find anything is to know the S3 path — which means the knowledge lives in people's heads.

Lesson 29 covers what the catalog has become, which is considerably more than a list of tables.

Governance

Lake Formation decides who can see which table, and which **columns** inside it, enforced consistently across every engine that reads through the catalog.

The swamp

The failure mode has a name. A **data swamp** is a lake with no catalog and no owner: the data is all there, and nobody can use any of it. Every file is technically retrievable and practically lost.

It is not caused by bad storage. It is caused by skipping the third and fourth pieces above because the first two were enough to demo.

When to use it

Use a lake for: - Raw data you must retain for years for audit or regulatory reasons - Schemas you do not control — SaaS exports, logs, third-party feeds - Data science and exploration on unmodelled data - Anything where "we might need it" is a real answer

Do not use it for: - Numbers the CFO signs off on. Those go through the warehouse.

In practice

On our platforms: - `raw/` and `bronze/` prefixes on S3 are the lake. - **Every file lands immutable, exactly once.** Nothing is ever edited in place. - Re-reading is cheap, which is why we can afford to reprocess history when a rule changes.

L03 · The Data Lake

The property to internalise: *cheap to keep, cheap to re-read.* Both halves matter. Storage you cannot afford to re-read is an archive, not a lake.

Checklist

- ☐ I can explain schema-on-read and name one risk it creates
- ☐ I can name all four components of a real lake, not just S3
- ☐ I can explain what a data swamp is and what causes it
- ☐ I know which prefixes on our platform are the lake
- ☐ I understand why files are never edited in place

You've got it when you can…

…be shown an S3 bucket full of Parquet and say precisely what is missing before it can honestly be called a data lake — and what will go wrong if those pieces are never added.

MODULE 0 · LESSON 04

The Lakehouse

Lake prices with warehouse behaviour. One open table format on top of ordinary files is the whole trick.

1 · WHAT IT IS

A metadata layer that makes a pile of files behave like a table

The files do not change. What changes is that something now tracks which files belong to the table right now.

AWS: S3 Tables (Iceberg)

2 · HOW IT WORKS

WHAT PLAIN FILES CANNOT DO — AND WHAT ICEBERG ADDS

THE PROBLEM — `files alone`

Files on S3 cannot UPDATE, cannot roll back, and two writers at once corrupt each other.

TABLE FORMAT — `Apache Iceberg v2`

A metadata tree over the same Parquet files: ACID commits, schema evolution, row-level MERGE.

SNAPSHOTS — `time travel`

Every write creates a snapshot, so you can query the table exactly as it was this morning.

MANY ENGINES — `Athena · Redshift · EMR · Glue`

Every engine reads the same table through the catalog. No copies means no drift between them.

```
same files as the lake · same guarantees as the warehouse · one copy of the data
```

3 · WHEN TO USE IT

You need updates and deletes on lake data

More than one engine reads the same table

You want one copy, not five

NOT FOR

Small data — a plain database is simpler

4 · IN PRACTICE

S3 Tables is AWS-managed Iceberg.

Bronze and silver are Iceberg tables.

Loads MERGE on a key, never blind append.

this is what we build

IN PLAIN ENGLISH

The garage after someone fitted shelves, labels and a lock. Same cheap space — now you can find, change and trust what is in it.

MODULE 0 · LESSON 04

L04 · The Lakehouse

Slide: `_render/L04-lakehouse.html`

What it is

A lakehouse gives you lake economics with warehouse behaviour. The bridge that makes it possible is an **open table format** — a metadata layer that sits on top of ordinary files and makes a pile of them behave like a table.

The files do not change. What changes is that something now tracks *which files belong to the table right now*, and can change that answer atomically.

On AWS: Amazon S3 Tables (managed Apache Iceberg) + Glue Data Catalog + Redshift + Athena + EMR.

How it works

The problem it solves

Plain files on S3 cannot do three things a database takes for granted:

1. **Update a row.** Objects are replaced whole, never edited.
2. **Roll back.** Once you have overwritten a file, the previous state is gone.
3. **Two writers at once.** Concurrent writes to the same prefix corrupt each other; there is no transaction.

Every one of those is fatal for a table people report from.

What the table format adds

Apache Iceberg keeps a metadata tree beside the data files. That gives you:

- **ACID commits** — a write either becomes visible in its entirety, or not at all
- **Schema evolution** — add, rename or drop a column without rewriting the data
- **Row-level updates and deletes** — Iceberg v2 uses *delete files* rather than rewriting whole data files
- **Snapshots and time travel** — query the table as it was at any earlier point

Many engines, one copy

Athena, Redshift, EMR and Glue all read the same Iceberg table through the catalog. There is no export step, no nightly copy, and therefore no drift between what two teams see.

This is the property people underestimate. Most "our numbers do not match" incidents are two copies of the same data that diverged.

When to use it

Use a lakehouse when: - You need updates and deletes on data that lives in the lake - More than one engine reads the same table - You want one copy of the truth, not five extracts

Do not use it when: - The data is small. A plain database is simpler and you will be happier.

In practice

On our platforms: - **S3 Tables** is AWS-managed Iceberg — AWS handles compaction and snapshot maintenance that you would otherwise operate yourself. - **Bronze and silver are Iceberg tables**, not raw files. - Loads **MERGE on a key**. They never blindly append, which is what makes re-running a day safe.

L04 · The Lakehouse

Checklist

- ☐ I can name the three things plain files cannot do
- ☐ I can explain what a table format adds, without saying "it's like a database"
- ☐ I know what a snapshot is and what time travel is for
- ☐ I can explain why one copy read by many engines beats many extracts
- ☐ I know which layers of our platform are Iceberg

You've got it when you can…

…explain to someone who understands S3 but not Iceberg why adding a metadata layer over unchanged Parquet files is enough to give you transactions, updates and rollback — and why that was worth building.

MODULE 0 · LESSON 05

Data Mesh Is Not A Product

Mesh is about who owns the data, not where it sits. You can build one on a lakehouse — you cannot buy one.

1 · WHAT IT IS

An operating model for organisations, wearing the costume of an architecture

Nothing in a mesh is a service you can switch on. Every part of it is a decision about teams, ownership and accountability.

no service does this for you

2 · HOW IT WORKS

FOUR PRINCIPLES — ALL FOUR ARE ABOUT PEOPLE

DOMAIN OWNERSHIP — `org, not tech`

The team that produces the data owns it — its quality, its documentation and its pager.

DATA AS A PRODUCT — `SLA + contract`

Each dataset has a named owner, a published schema and consumers allowed to complain.

FEDERATED GOVERNANCE — `AWS Lake Formation`

A central group sets the rules; each domain applies them. Nobody becomes the bottleneck.

SELF-SERVE PLATFORM — `the platform is a product`

Domains publish data without filing a ticket, because the tooling is built for them to use.

```
on AWS: Lake Formation grants over RAM · Redshift datashares · federated catalogs
```

3 · WHEN TO USE IT

Many domains, each with its own engineers
The central team has become the bottleneck
Governance can be automated, not policed

NOT FOR

One team and one platform — pure overhead

4 · IN PRACTICE

Neither of our platforms is a mesh.
One platform team owns end to end.
Learn the word so you can push back on it.

mesh too early buys ceremony

IN PLAIN ENGLISH

Mesh is not a building. It is the rule about who holds the keys to each room and who you call when the lights are off.

L05 · Data Mesh Is Not A Product

Slide: `_render/L05-data-mesh.html`

What it is

Data mesh is an **operating model for organisations**, wearing the costume of an architecture. It is the most over-used and least-understood of the four words, and you will meet it in RFPs written by people who could not implement it.

Nothing in a mesh is a service you can switch on. Every part of it is a decision about teams, ownership and accountability. You can build a mesh *on* a lakehouse. You cannot buy one.

The four principles — all four are about people

1. Domain ownership

The team that **produces** the data owns it — its quality, its documentation, and its pager. Not a central data team that inherited a pipeline from a consultant three years ago.

2. Data as a product

Each dataset has a named owner, a published schema, a stated SLA, and consumers who are entitled to complain when it breaks. "Product" is not a metaphor here; it means someone is accountable for whether it is any good.

3. Federated governance

A central group sets the rules — naming, classification, PII handling, retention — and each domain applies them to its own data. Nobody has to queue behind a central team to publish.

4. Self-serve platform

Domains publish data without filing a ticket, because the platform itself is built as a product for them to use. If publishing requires a platform engineer, you do not have a mesh; you have a bottleneck with extra ceremony.

How AWS actually implements the sharing

The organisational parts are yours to build. The technical sharing is well supported:

NEED	AWS MECHANISM
Share catalog tables across accounts	**Lake Formation** cross-account grants over **AWS RAM**
Grant by classification rather than by name	**LF-TBAC** — tag-based access control
Make a shared database appear locally	**Resource links**
Share live warehouse data	**Redshift datashares** — across clusters, accounts and Regions
Expose a domain's system without copying it	**Federated catalogs** (Lesson 29)

Two grant styles are worth knowing apart: **named-resource** grants name specific tables, and **tag-based** grants apply to anything carrying a tag. Tag-based scales; named-resource does not.

When it makes sense

A mesh pays off when: - There are many domains, each with its own engineers who can own data - The central data team has genuinely become the bottleneck - Governance can be automated and enforced, not manually policed

L05 · Data Mesh Is Not A Product

It does not pay off when: - There is one team and one platform. Then it is pure overhead — meetings, standards documents and ownership matrices with no autonomy to enable.

In practice

Neither Tamimi nor Apparel Group is a mesh, and neither needs to be. One platform team owns ingestion, transformation, the warehouse and the reporting layer end to end.

Learn the word properly anyway — because someone will propose a mesh in a steering meeting, and the useful response is not "no", it is *"which domain teams would own which products, and who is on call for them?"* That question resolves the conversation quickly and honestly.

Checklist

- ☐ I can state the four principles and say why each is organisational
- ☐ I can explain why you cannot buy a mesh
- ☐ I know which AWS mechanisms implement cross-domain sharing
- ☐ I know the difference between named-resource and tag-based grants
- ☐ I can say why our platform is not a mesh, without being dismissive

You've got it when you can…

…hear "we should build a data mesh" in a meeting and ask the two questions that establish whether the organisation is actually shaped for one — without either agreeing reflexively or dismissing it.

MODULE 0 · LESSON 06

So Which One Do You Need?

Four options side by side. For almost every retail business the honest answer is the same one.

1 · THE QUESTION

You are not choosing a favourite — you are choosing what to pay for

Every option buys one thing and gives up another. Name what you are buying before you name the technology.

2 · SIDE BY SIDE

WAREHOUSE

Schema on write, modelled first
Fast joins, numbers that reconcile
Rigid — changes cost a migration
Priced per hour of compute

ON AWS
`Amazon Redshift`

BUY: TRUST

LAKE

Schema on read, decide later
Keeps everything, costs very little
No updates, no guarantees
Priced per terabyte scanned

ON AWS
`S3 + Glue + Athena`

BUY: OPTIONALITY

LAKEHOUSE

Lake storage, warehouse behaviour
ACID, MERGE, time travel
Many engines read one copy
More moving parts to operate

ON AWS
`S3 Tables + Redshift`

BUY: BOTH

MESH

An ownership model, not a store
Domains own their own products
Needs many teams to pay off
Sits on top of one of the others

ON AWS
`Lake Formation + RAM`

BUY: AUTONOMY

One platform team, eight sources, retail reporting → build the lakehouse. Revisit mesh when you have five domain teams and a queue.

IN PLAIN ENGLISH

Do not pick the architecture with the best conference talks. Pick the one whose trade-off you are actually willing to live with.

L06 · So Which One Do You Need?

Slide: `_render/L06-which-one.html`

The question

You are not choosing a favourite. You are choosing **what to pay for**, because every option buys one thing and gives up another.

Name what you are buying before you name the technology. Teams that do it the other way round end up defending a choice they made for reasons they cannot articulate.

Side by side

	WAREHOUSE	LAKE	LAKEHOUSE	MESH
Schema	on write, modelled first	on read, decide later	lake storage, warehouse behaviour	n/a — it is an ownership model
Strength	fast joins, numbers reconcile	keeps everything, costs little	ACID, MERGE, time travel	domains own their products
Weakness	rigid; change costs a migration	no updates, no guarantees	more moving parts to operate	needs many teams to pay off
Priced	per hour of compute	per TB scanned	both, depending on engine	sits on top of one of the others
On AWS	Amazon Redshift	S3 + Glue + Athena	**S3 Tables + Redshift**	Lake Formation + RAM
You buy	trust	optionality	both	autonomy

The honest recommendation

For a retail group with **one platform team, eight source systems, and reporting as the primary use case**: build the **lakehouse**.

The reasoning, stated plainly:

- A pure warehouse forces you to decide the meaning of every field before you have seen the data. With eight unfamiliar source systems, you will be wrong, and every correction is a migration.
- A pure lake cannot give the finance team a number they can sign off on, because nothing enforces the shape.
- A lakehouse lets you land everything cheaply and *then* model the part that needs modelling, without moving the data to do it.
- A mesh solves a coordination problem you do not have yet. Revisit it when you have five domain teams and a genuine queue for the central team.

The mistake to avoid

Do not pick the architecture with the best conference talks. Pick the one whose **trade-off you are willing to live with on a Tuesday afternoon in month nine**.

Every one of these options has a bad day built into it: - The warehouse's bad day is a schema migration nobody scoped. - The lake's bad day is discovering the file format changed upstream two months ago. - The lakehouse's bad day is compaction, snapshot expiry and a catalog that has to be kept correct. - The mesh's bad day is a domain team that stopped answering.

Choose the bad day you can staff.

L06 · So Which One Do You Need?

Checklist

- ☐ I can fill in the comparison table from memory
- ☐ I can state what each option *buys* in one word
- ☐ I can give the honest recommendation for our situation and defend it
- ☐ I can name the characteristic bad day for each option
- ☐ I know what would have to change for a mesh to become the right answer

You've got it when you can…

…sit in a design review, hear four people advocate four different architectures, and reframe the argument from "which is best" to "which trade-off are we willing to operate" — then land the decision.

MODULE 0 · LESSON 07

Facts, Dimensions and Grain

The one modelling skill that transfers to every warehouse you will ever touch. It starts with a sentence, not a diagram.

1 · WHAT IT IS

A star schema: one fact table, surrounded by the dimensions you slice it by

Facts are what happened, and they are measured. Dimensions are the context. Almost every retail reporting model is this shape.

2 · HOW IT WORKS

SAY THE GRAIN OUT LOUD BEFORE YOU WRITE ANY DDL

GRAIN — `decide this first`

"One row = one line on one receipt." Every other decision follows from that sentence.

FACT TABLE — `fct_sales_line`

Long and narrow: numbers you can add up, plus keys to the dimensions. Grows forever.

DIMENSION — `dim_store · dim_product`

Short and wide: the attributes you filter and group by. Changes slowly — hence Lesson 08.

MEASURES — `not all numbers add up`

Additive sums anywhere · semi-additive not over time (stock) · non-additive never (ratios).

```
star = 1 fact + many dims  ·  snowflake = dims normalised further  ·  prefer star
```

3 · WHEN TO USE IT

Any reporting model, on any warehouse
When business users write their own SQL
When one measure is sliced many ways

NOT FOR

Machine-learning feature tables

4 · IN PRACTICE

Gold layer only. Bronze and silver keep the source shape, unmodelled.
One conformed dim_date across all facts.

grain first · columns second

IN PLAIN ENGLISH

The fact table is the receipt. The dimensions are the shop, the till, the product and the day. You count receipts and slice by the rest.

L07 · Facts, Dimensions and Grain

Slide: `_render/L07-dimensional-modelling.html`

What it is

A **star schema**: one fact table in the middle, surrounded by the dimensions you slice it by.

Facts are *what happened*, and they are measured. Dimensions are *the context* you filter and group by. Almost every retail reporting model in the world is this shape, and once you can see it you will see it everywhere.

Say the grain out loud, first

Before any DDL, finish this sentence:

> **"One row in this table equals one _____."**

For example: *one row = one line on one receipt.*

Everything else follows. The grain decides which measures are valid, which dimensions can attach, and whether a number can be summed. Teams that skip this step spend the next year discovering that their fact table means slightly different things in different places.

Two rules about grain: - **Go as fine as you can afford.** You can always aggregate up; you can never disaggregate down. - **One grain per fact table.** Mixing "one row = one receipt line" with "one row = one receipt" in the same table guarantees double counting.

Facts

Long and narrow. Contains: - **Measures** — numbers you do arithmetic on - **Foreign keys** — to the dimensions - **Degenerate dimensions** — identifiers like receipt number that have no attributes of their own

Fact tables grow forever. They are where your storage and your compute go.

Dimensions

Short and wide. Descriptive attributes you filter and group by: store name, region, product category, brand, day of week.

Dimensions change slowly — which is a whole lesson of its own (Lesson 08).

Conformed dimensions are dimensions shared across several fact tables. One `dim_date`, one `dim_store`, used by sales, stock and footfall alike. Without them, two marts that both report "sales by region" will disagree, and the disagreement will be structural rather than a bug you can find.

Measures — not all numbers add up

TYPE	ADDS UP OVER…	EXAMPLE
Additive	every dimension	sales amount, quantity
Semi-additive	everything *except* time	stock on hand, account balance
Non-additive	nothing	ratios, percentages, unit price

Summing a semi-additive measure over time is the most common wrong number in retail analytics. Stock on hand of 100 on Monday and 100 on Tuesday is not 200.

For non-additive measures, store the **numerator and denominator** as additive columns and compute the ratio at query time. Never store the ratio and sum it.

L07 · Facts, Dimensions and Grain

Star vs snowflake

- **Star** — dimensions are flat and denormalised. More storage, simpler queries, faster joins.
- **Snowflake** — dimensions are normalised into sub-tables. Less storage, more joins, harder for business users.

Prefer star. Storage is cheap; the confusion that snowflaking causes among self-service users is not.

When to use it

Use dimensional modelling for: any reporting model, on any warehouse; anywhere business users write their own SQL; anywhere one measure is sliced many ways.

Do not use it for: machine-learning feature tables, which want one wide row per entity rather than a star.

In practice

- Dimensional modelling applies to the **gold layer only**. Bronze and silver keep the source shape.
- **One conformed** `dim_date` across every fact table, generated once.
- The grain of each fact is written into the model documentation, not just implied by the code.

Checklist

- ☐ I can state the grain of a table as a sentence before writing DDL
- ☐ I can tell a fact from a dimension without being told
- ☐ I can classify a measure as additive, semi-additive or non-additive
- ☐ I know why ratios are stored as numerator and denominator
- ☐ I can explain what a conformed dimension prevents
- ☐ I default to star, and can say why

You've got it when you can…

…be handed a source table you have never seen, ask three questions, and produce a fact-and-dimension design with the grain stated in one sentence that survives review.

MODULE 0 · LESSON 08

When A Dimension Changes

A store moves to a new region on 1 March. Do last year's sales belong to the old region or the new one? That answer is the SCD type.

1 · WHAT IT IS

A business question wearing a technical name

"Slowly changing dimension" just means: an attribute changed, and someone has to decide what happens to the history.

ask the business, not the DBA

2 · HOW IT WORKS

THREE ANSWERS — AND ONE OF THEM IS THE DEFAULT

TYPE 1 · OVERWRITE — `history destroyed`

Update the row in place. The old region never existed. Fine for typos, wrong for reporting.

TYPE 2 · NEW ROW — `valid_from · valid_to · is_current`

Close the old row, insert a new one. History preserved. This is the retail default.

TYPE 3 · EXTRA COLUMN — `one step only`

Keep prev_region beside region. Remembers exactly one change. Rarely enough in practice.

SURROGATE KEY — `store_sk, not store_id`

The fact points at one version of the row. Without this, Type 2 cannot work at all.

```
the fact joins on store_sk (the version) — never on store_id (the business key)
```

3 · WHEN TO USE WHICH

Type 2 — default for anything reported on
Type 1 — genuine corrections and typos
Type 3 — almost never; ask why first

NOT FOR

Fast-churning attributes — split them out

4 · IN PRACTICE

dim_store and dim_product are Type 2.
dbt snapshots build the history for you.
BI filters on is_current = true.

get this wrong and last year moves

IN PLAIN ENGLISH

Did the shop move, or is it a different shop now? Type 1 says it moved. Type 2 says there are two shops, and last year belongs to the first.

L08 · When A Dimension Changes

Slide: `_render/L08-slowly-changing-dimensions.html`

The question

A store moves from the Central region to the Western region on 1 March.

Someone runs "sales by region for last year". Do that store's January sales appear under **Central** (where it was at the time) or **Western** (where it is now)?

Both answers are defensible. Both are wanted by different people. **The choice is the SCD type**, and it is a business decision, not a technical one — which is why the first thing to do is ask the business, not the DBA.

Type 1 — overwrite

Update the row in place. The old region never existed; every historical report retrospectively moves the store to Western.

- **Use for:** genuine corrections. A misspelled store name, a wrong postcode.
- **Do not use for:** anything reported on over time. It silently rewrites history.

Type 2 — new row

Close the current row, insert a new one. The dimension gains three columns:

COLUMN	MEANING
`valid_from`	when this version became true
`valid_to`	when it stopped (or a far-future date / null)
`is_current`	convenience flag for "the version now"

History is preserved: January sales stay attached to the Central version of the store, and March onwards attaches to the Western version.

This is the retail default. Assume Type 2 unless someone argues otherwise.

Type 3 — extra column

Keep `prev_region` alongside `region`. Remembers exactly one change, forever.

Rarely enough in practice. If someone asks for Type 3, ask what happens on the second change — the answer is usually "oh".

The surrogate key is what makes Type 2 work

This is the part people miss, and getting it wrong quietly breaks everything.

- `store_id` is the **business key** — stable, from the source system. Store 4471 is always store 4471.
- `store_sk` is the **surrogate key** — one per *version* of the row. Store 4471 has one `store_sk` for its Central period and a different one for its Western period.

The fact table joins on `store_sk`, never on `store_id`.

If the fact joins on the business key, every historical row silently picks up whichever version is current — which is Type 1 behaviour with Type 2 storage costs. You have built the complexity and got none of the benefit.

When to use which

SITUATION	TYPE
Anything reported on over time	**Type 2** — the default
Fixing a genuine data error	Type 1
One attribute, one step of history, strong reason	Type 3
Attribute changes very frequently	Neither — split it into its own table or make it a fact

L08 · When A Dimension Changes

That last row matters: a "slowly changing" dimension that changes daily is not slowly changing. Modelling it as Type 2 produces an enormous dimension. Move the volatile attribute out.

In practice

- `dim_store` and `dim_product` are **Type 2**.
- **dbt snapshots** build the history — you declare the key and the columns to track, and dbt maintains `valid_from` / `valid_to` for you.
- BI filters on `is_current = true` for "as it is now" reporting, and joins through `store_sk` for "as it was" reporting.

Checklist

- ☐ I can state the business question that decides the SCD type
- ☐ I know the three types and when each applies
- ☐ I can explain why a surrogate key is required for Type 2
- ☐ I would notice a fact table joining on a business key in review
- ☐ I know what to do with a fast-changing attribute
- ☐ I know which of our dimensions are Type 2 and how they are built

You've got it when you can…

…be told "the store moved region" and immediately ask the business the right question — then implement whichever answer they give, and explain to a reviewer why the fact table joins where it does.

MODULE 0 · LESSON 09

Three Schools Of Warehouse Design

You will meet all three names in interviews and RFPs. They disagree on where the modelling effort goes, not on the goal.

1 · WHAT IT IS

Same destination, three different orders of work

All three end with business users querying trustworthy tables. They differ on what you build first and how much you model up front.

2 · SIDE BY SIDE

KIMBALL — bottom-up

Build the marts first, one business process at a time
Star schemas, made for querying
Fast to first value; users understand it
Risk: marts drift apart without conformed dimensions

ON AWS

`dbt marts on Redshift`

WHAT LAKEHOUSES USE

INMON — top-down

One normalised 3NF core for the enterprise
Marts are derived from that core
Single version of truth by construction
Slow to first value; big up-front modelling effort

ON AWS

`3NF core in Redshift`

LARGE ENTERPRISES

DATA VAULT — audit-first

Hubs (keys), links (relationships), satellites (attributes)
Built for change and full auditability
Every source keeps its own history
Very many tables; needs generation and tooling

ON AWS

`generated dbt models`

HEAVY REGULATION

Modern lakehouses come out Kimball-shaped at the gold layer — because bronze and silver already hold the raw history the other two schools model explicitly.

IN PLAIN ENGLISH

Kimball furnishes one room at a time. Inmon draws the whole house first. Data Vault keeps a receipt for every brick. We furnish rooms.

MODULE 0 · LESSON 09

L09 · Three Schools Of Warehouse Design

Slide: `_render/L09-three-schools.html`

What it is

Three named schools of warehouse design. You will meet all three in interviews, RFPs and consultancy decks. They agree on the destination — business users querying trustworthy tables — and disagree about **what you build first and how much you model up front**.

Kimball — bottom-up

Build the **marts first**, one business process at a time. Sales this quarter, stock next quarter. Each mart is a star schema, designed for querying.

- **Strength:** fast to first value, and business users can understand the model
- **Weakness:** marts drift apart unless you enforce **conformed dimensions** across them
- **On AWS:** dbt marts on Redshift

Inmon — top-down

Build one **normalised 3NF core** for the whole enterprise first. Marts are then derived from that core.

- **Strength:** single version of truth by construction, not by discipline
- **Weakness:** slow to first value; needs a large modelling effort before anyone sees a dashboard
- **On AWS:** a 3NF core schema in Redshift, marts built from it

Data Vault — audit-first

Model as **hubs** (business keys), **links** (relationships) and **satellites** (attributes with history).

- **Strength:** built for change and full auditability; every source keeps its own lineage and history
- **Weakness:** a great many tables; genuinely needs code generation and strong tooling
- **On AWS:** generated dbt models

Which one we use, and why

Modern lakehouses come out **Kimball-shaped at the gold layer**.

The reason is structural, not fashion: bronze and silver already hold the raw, historical, source-shaped data that Inmon's 3NF core and Data Vault's satellites exist to preserve. The lake does that job now. So the warehouse layer is free to be purely about *making questions easy to ask* — which is exactly what Kimball is for.

Put another way: the medallion architecture already gave you the top-down layer. Building a second one inside the warehouse is paying twice.

When the other two are right

- **Inmon** — a very large enterprise with many source systems feeding one regulated definition of a customer or a product, where the cost of inconsistency is higher than the cost of delay.
- **Data Vault** — heavy regulation, where you must be able to prove where any value came from and what it was at any past moment, across many changing sources.

Both are legitimate. Neither is what a retail group with eight sources and a reporting mandate needs.

L09 · Three Schools Of Warehouse Design

Checklist

- ☐ I can describe each school in one sentence
- ☐ I can say what each optimises for and what it costs
- ☐ I can explain why lakehouses land on Kimball at the gold layer
- ☐ I can name a situation where Inmon or Data Vault would be right
- ☐ I would not be intimidated by a vendor deck built on one of them

You've got it when you can…

…hear a consultant propose Data Vault, understand what they are actually optimising for, and either agree with a reason or decline with one — rather than nodding because the diagram looked authoritative.

MODULE 0 · LESSON 10

Creating Tables In Redshift

The DDL looks like PostgreSQL. Four things behave completely differently — and the last one catches every SQL developer.

1 · WHAT IT IS

In a warehouse, physical layout is part of the schema

In an app database the engine hides where rows live. In Redshift you choose it, and that choice is most of your query performance.

2 · HOW IT WORKS

FOUR DECISIONS YOU MAKE IN EVERY CREATE TABLE

DISTRIBUTION — `DISTSTYLE / DISTKEY`

How rows spread across slices. KEY co-locates joins · ALL copies small dims everywhere · AUTO decides.

SORT KEY — `SORTKEY(sale_date)`

Physical order on disk. Redshift skips whole blocks that cannot satisfy your WHERE clause.

ENCODING — `ENCODE az64 · zstd`

Per-column compression. AUTO is good; choosing it deliberately matters on very large facts.

CONSTRAINTS ARE HINTS — `not enforced`

PRIMARY KEY and FOREIGN KEY are informational only. Redshift will happily store duplicates.

```
CREATE TABLE fct_sales (...) DISTKEY(store_sk) SORTKEY(sale_date);
```

3 · RULES OF THUMB

DISTKEY on the biggest join column

DISTSTYLE ALL for small dimensions

SORTKEY on what you filter — usually date

NEVER

Assume a PRIMARY KEY stops duplicates

4 · IN PRACTICE

dbt config sets dist and sort per model.

Facts: DISTKEY on the busiest foreign key.

Dimensions under ~5M rows: DISTSTYLE ALL.

test uniqueness in dbt, not in DDL

IN PLAIN ENGLISH

Redshift's PRIMARY KEY is a note to the query planner, not a lock on the door. If you need uniqueness, you have to test for it yourself.

L10 · Creating Tables In Redshift

Slide: `_render/L10-building-in-redshift.html`

What it is

Redshift's DDL looks like PostgreSQL, which is exactly why people get caught out. In a warehouse, **physical layout is part of the schema**. In an application database the engine hides where rows live; in Redshift you choose it, and that choice is most of your query performance.

Four decisions in every `CREATE TABLE`.

1. Distribution — how rows spread across slices

STYLE	BEHAVIOUR	USE FOR
`DISTSTYLE KEY`	rows with the same key land on the same slice	large fact tables, on the busiest join column
`DISTSTYLE ALL`	a full copy on every node	small dimensions (roughly under a few million rows)
`DISTSTYLE EVEN`	round-robin	staging tables, or when nothing joins
`DISTSTYLE AUTO`	Redshift chooses and may change it	the sane default until you have a measured reason

The point of `KEY` is **co-location**: if the fact and the dimension are distributed on the same key, the join happens on each slice locally instead of shuffling data across the network.

2. Sort key — physical order on disk

Redshift stores per-block min/max values. If a block cannot possibly satisfy your `WHERE` clause, it is skipped without being read.

Sort on **what you filter by** — for almost all retail facts, that is the date. A fact table sorted by date and filtered by date reads only the relevant blocks.

3. Encoding — per-column compression

`ENCODE AUTO` is genuinely good. Choosing deliberately (`az64` for numerics and dates, `zstd` for text) matters on very large fact tables, where the difference is measured in terabytes scanned.

4. Constraints are hints — the one that catches everyone

```
CREATE TABLE dim_store (
  store_sk   BIGINT PRIMARY KEY,   -- <- NOT enforced
  store_id   VARCHAR(20),
  ...
);
```

`PRIMARY KEY`, `UNIQUE` and `FOREIGN KEY` are informational only in Redshift. They inform the query planner. They do **not** prevent you inserting duplicates.

This surprises every SQL developer in the room, and it is the source of a whole family of production bugs: a load runs twice, duplicates appear, the constraint says nothing, and a dashboard doubles.

The consequence: if you need uniqueness, you must *test* for it. In dbt that is a `unique` test on the model. Not a constraint, a test.

L10 · Creating Tables In Redshift

Worked shape

```
CREATE TABLE fct_sales_line (
  sale_date     DATE       NOT NULL,
  store_sk      BIGINT     NOT NULL,
  product_sk    BIGINT     NOT NULL,
  receipt_no    VARCHAR(32),
  quantity      DECIMAL(12,3),
  net_amount    DECIMAL(14,2),
  vat_amount    DECIMAL(14,2)
)
DISTKEY (store_sk)
SORTKEY (sale_date);
```

Rules of thumb

- **DISTKEY** on the biggest join column of the biggest table
- **DISTSTYLE ALL** for small dimensions — the copy is cheaper than the shuffle
- **SORTKEY** on what you filter by, which is usually date
- **Never** assume a `PRIMARY KEY` stops duplicates

In practice

- dbt config sets `dist` and `sort` per model, so the physical design lives beside the SQL and is reviewed with it.
- Facts get `DISTKEY` on the busiest foreign key.
- Dimensions under roughly 5M rows get `DISTSTYLE ALL`.
- **Uniqueness is tested in dbt, not declared in DDL.**

Checklist

- ☐ I can explain all four distribution styles and when each applies
- ☐ I can explain block skipping and why sort key matters
- ☐ I know that constraints are not enforced, and what I do instead
- ☐ I can write a `CREATE TABLE` for a fact with sensible dist and sort
- ☐ I know where dist and sort live in our dbt models

You've got it when you can…

…look at a slow Redshift query, name which of the four decisions was made wrong, and predict what changing it would do — before you run the experiment.

MODULE 0 · LESSON 11

Getting Rows In, Correctly

Row-by-row INSERT is how you turn a warehouse into a very expensive slow database. Four patterns do it properly.

1 · WHAT IT IS

A warehouse wants whole batches, not individual rows

Columnar storage rewrites blocks on every write. One INSERT of a million rows is cheap; a million INSERTs of one row is not.

never INSERT row by row

2 · HOW IT WORKS

THE FOUR-STEP LOAD THAT YOU CAN SAFELY RE-RUN

COPY — `COPY ... FROM 's3://...'`

The bulk path: many files read in parallel, one per slice. Orders of magnitude faster than INSERT.

STAGE FIRST — `stg_ tables`

Land into a staging table, validate it there, then move. Never write half a load into a live table.

MERGE — `MERGE INTO ... ON key`

Insert-or-update on a key in one statement. This is what makes a load safe to run twice.

ONE TRANSACTION — `BEGIN ... COMMIT`

Wrap staging into target so a reader never sees the table half-loaded. All of it, or none of it.

```
re-running yesterday must produce exactly yesterday's table — that is idempotency
```

3 · RULES OF THUMB

COPY for bulk · MERGE for upsert

Always stage, then merge or swap

One transaction per load, not per row

NEVER

Point an application at it as an OLTP store

4 · IN PRACTICE

dbt incremental models emit the MERGE.

unique_key is the merge key.

Re-running a day is safe, and expected.

idempotent, or it is not done

IN PLAIN ENGLISH

Unload the whole lorry at the bay, check it, then move it to the shelves. Do not carry items in one at a time through the front door.

L11 · Getting Rows In, Correctly

Slide: `_render/L11-loading-a-warehouse.html`

What it is

Row-by-row `INSERT` is how you turn a warehouse into a very expensive, very slow database.

The reason is columnar storage. Every write touches and rewrites blocks. One `INSERT` of a million rows is cheap; a million `INSERT`s of one row is catastrophic. Application developers arrive with the opposite instinct, so this needs saying out loud.

Four patterns do it properly.

1. COPY — the bulk path

```
COPY stg_sales_line
FROM 's3://bucket/staging/sales_line/dt=2026-08-11/'
IAM_ROLE 'arn:aws:iam::...:role/redshift-loader'
FORMAT AS PARQUET;
```

Reads many files in parallel, one per slice. Orders of magnitude faster than `INSERT`, and the reason your landing job should produce *several* files rather than one enormous one.

`COPY` can also read from EMR, DynamoDB and a remote host over SSH — but S3 is the path you will use.

2. Stage first

Land into a **staging table**, validate it there, then move it. Never write half a load into a table someone is querying.

Staging also gives you somewhere to run your checks — row counts, null keys, unexpected values — while there is still an easy way to abandon the load.

3. MERGE — insert or update on a key

```
MERGE INTO fct_sales_line AS t
USING stg_sales_line AS s
ON t.merge_key = s.merge_key
WHEN MATCHED THEN UPDATE SET ...
WHEN NOT MATCHED THEN INSERT ...;
```

This is what makes a load **safe to run twice**. Without it, re-running yesterday's load doubles yesterday's data, and you find out from a dashboard.

4. One transaction

Wrap the move from staging into target in a single transaction so a reader never sees the table half-loaded. All of it, or none of it.

Idempotency — the property that matters most

> **Re-running yesterday's load must produce exactly yesterday's table.**

Everything above exists to make that true. It matters because failures are normal: a source is late, a job dies mid-run, someone needs a backfill. If re-running is dangerous, every incident becomes a manual repair. If re-running is safe, incidents become a retry.

Test it deliberately: run a load twice on purpose and diff the result. Do it at table two, not table fifty.

L11 · Getting Rows In, Correctly

Rules of thumb

- `COPY` for bulk, `MERGE` for upsert
- Always stage, then merge or swap
- One transaction per load, not per row
- **Never** point an application at the warehouse as an OLTP store

Maintenance

`VACUUM` reclaims space and re-sorts; `ANALYZE` refreshes statistics for the planner. Redshift automates much of this now, and Serverless more so — but "automated" is not "absent". Know that they exist, and check they are keeping up on your largest tables.

In practice

- **dbt incremental models emit the** `MERGE` for you.
- `unique_key` in the model config is the merge key.
- Re-running a day is safe and expected — that is a design property we rely on, not a lucky accident.

Checklist

- ☐ I can explain why row-by-row INSERT is wrong in a warehouse
- ☐ I can write a COPY from S3 and say why parallel files matter
- ☐ I always stage before touching a live table
- ☐ I can write a MERGE and explain what makes it idempotent
- ☐ I have actually tested that re-running a load is safe
- ☐ I know where `unique_key` is set in our dbt models

You've got it when you can…

…be told at 7am that last night's load ran twice, check the merge key, and say with confidence whether there is a problem — rather than starting a manual reconciliation.

MODULE 0 · LESSON 12

Materialized Views

A view that stores its answer. The warehouse's cache — and, later in this module, the landing point for streaming data.

1 · WHAT IT IS

A view re-runs the query · a materialized view keeps the result

That single difference turns a thirty-second dashboard query into a sub-second one — and hands you a cache-invalidation problem.

2 · HOW IT WORKS

FOUR THINGS TO KNOW BEFORE YOU CREATE ONE

IT IS A TABLE — `stored, not virtual`
The result occupies storage and is as stale as its last refresh. Treat it as derived data.

AUTO REFRESH — `AUTO REFRESH YES`
Redshift can refresh the view itself when the base tables change, instead of you scheduling it.

INCREMENTAL — `not always possible`
Where the query allows it, only changed rows are reprocessed rather than the whole result.

OVER THE LAKE — `Redshift Spectrum`
An MV can sit on external data lake tables, caching an expensive S3 scan inside the warehouse.

```
Lesson 23: this same object is how Kinesis and Kafka data lands inside Redshift
```

3 · WHEN TO USE IT

Expensive aggregations queried all day
Dashboard queries that repeat unchanged
Caching slow external-table scans

NOT FOR

Data that must be exactly current

4 · IN PRACTICE

Power BI reads MVs, not raw fact tables.
Refresh runs at the end of the gold build.
A stale MV is a wrong dashboard: alarm it.

a cache you must invalidate

IN PLAIN ENGLISH

A view is a recipe cooked again on every order. A materialized view is the dish already made and kept warm — fast, if someone remakes it.

L12 · Materialized Views

Slide: `_render/L12-materialized-views.html`

What it is

A **view** re-runs its query every time you select from it. A **materialized view** stores the result and refreshes it.

That single difference turns a thirty-second dashboard query into a sub-second one — and hands you a cache-invalidation problem in exchange.

Four things to know before you create one

1. It is a table

The result occupies storage and is exactly as stale as its last refresh. Treat it as derived data with an age, not as a view that happens to be fast.

2. AUTO REFRESH

```
CREATE MATERIALIZED VIEW mv_daily_sales
AUTO REFRESH YES
AS SELECT ...;
```

Redshift can refresh the view itself when the base tables change, rather than you scheduling it. Convenient — and worth knowing about, because a view refreshing on its own is a source of compute you did not schedule.

3. Incremental refresh

Where the query shape allows it, Redshift reprocesses only the changed rows rather than recomputing the whole result. Not every query qualifies; complex ones fall back to a full refresh. If refresh cost matters, check which one you are getting.

4. It can sit over the data lake

A materialized view can be defined on **external data lake tables** read through Spectrum, with incremental maintenance. That is a genuinely useful pattern: cache an expensive S3 scan inside the warehouse, and let dashboards hit the cache.

Where this comes back

Lesson 23: the same object — a materialized view — is how Kinesis and Kafka data lands directly inside Redshift. Redshift streaming ingestion maps an MV straight onto a stream, with no S3 staging at all.

Worth flagging now so that when it appears in Part D it is a familiar object doing a new job, rather than a new concept.

When to use it

Use an MV for: - Expensive aggregations queried many times a day - Dashboard queries that repeat unchanged - Caching slow external-table scans

Do not use it for: - Data that must be exactly current at the moment of reading

The failure mode

A stale materialized view is a wrong dashboard, and it is wrong silently — the query succeeds, returns quickly, and gives yesterday's answer.

So: alarm on refresh age, not just on refresh failure. A refresh that stopped running three days ago and never errored is the dangerous case.

In practice

- Power BI reads MVs, not raw fact tables.
- Refresh runs at the end of the gold build, as an explicit step.
- Refresh age is monitored, because a silent stale view is worse than an outage.

L12 · Materialized Views

Checklist

- ☐ I can explain the difference between a view and a materialized view
- ☐ I know what AUTO REFRESH does and what it costs
- ☐ I know incremental refresh is conditional on query shape
- ☐ I know an MV can sit over Spectrum external tables
- ☐ I monitor refresh **age**, not just refresh success
- ☐ I know an MV is the landing point for streaming ingestion (Lesson 23)

You've got it when you can…

…be shown a dashboard returning yesterday's numbers with no errors anywhere, and check the materialized view's refresh age first — because you know that is the failure that does not announce itself.

MODULE 0 · LESSON 13

Everything That Touches Redshift

Twenty-three ways in and out. Blue writes, green reads. Photograph this slide — you will come back to it all year.

1 · THE PARTICIPATION MATRIX

WRITES INTO REDSHIFT

11 paths in

COPY	bulk load from S3, EMR, DynamoDB or a remote host
INSERT · MERGE · CTAS	ordinary SQL, run inside the warehouse
JDBC / ODBC	clients and Query Editor v2 — small volumes only
Redshift Data API	HTTP/SDK, no held connection — built for Lambda
Glue · Spark · EMR	the Redshift connector writes DataFrames straight in
AWS DMS	Redshift as a target: full load, then ongoing CDC
Amazon Data Firehose	stages to S3, then issues COPY for you
Streaming ingestion	an MV on Kinesis or MSK — no S3 staging at all
Zero-ETL integration	managed replication — no pipeline to build
Datashare + allow-writes	a consumer INSERTs into the producer's data
Amazon AppFlow	scheduled SaaS objects landed as tables

READS FROM REDSHIFT

8 paths out

JDBC / ODBC	humans, ad-hoc SQL, Query Editor v2
Redshift Data API	applications and Lambda, no driver
UNLOAD to S3	publish gold back to the lake as Parquet
Glue · Spark · EMR	read frames out for ETL and ML
Athena connector	Athena queries Redshift tables
Glue federated catalog	catalog engines read it, governed by LF
Power BI · QuickSight	the consumption layer
Redshift ML · SageMaker	training reads, inference writes back

REDSHIFT AS A CLIENT — IT READS OTHER SYSTEMS TOO

Spectrum	S3 external tables, through the catalog
Federated query	live RDS / Aurora PostgreSQL or MySQL
Datashare · Lambda UDF	another warehouse · a function mid-query

IN PLAIN ENGLISH

Redshift is not a locked box with one door. It is a building with eleven loading bays, eight exits — and windows it can see out of.

L13 · Everything That Touches Redshift

Slide: `_render/L13-participation-matrix.html`

What it is

Redshift is not a locked box with one door. It is a building with **eleven loading bays, eight exits, and windows it can see out of**.

Most people know two or three of these paths and assume the rest do not exist — which is how you end up building a Lambda that opens a JDBC connection, or exporting to CSV because nobody knew `UNLOAD` existed.

This is the full matrix. Blue writes, green reads.

Writes into Redshift — eleven paths

PATH	WHAT IT IS	USE IT FOR
`COPY`	bulk load from S3, EMR, DynamoDB or a remote host	the main load path — anything above a few million rows
`INSERT` · `MERGE` · `CTAS`	ordinary SQL inside the warehouse	transformations between warehouse tables
JDBC / ODBC	clients and Query Editor v2	manual fixes, small reference data — never bulk
Redshift Data API	HTTP/SDK, no held connection	Lambda, Step Functions, applications
Glue · Spark · EMR connector	writes DataFrames straight in	Spark ETL that ends in the warehouse
AWS DMS	Redshift as a target endpoint	full load, then continuous CDC (Lesson 22)
Amazon Data Firehose	stages to S3, then issues `COPY`	streaming, with no code
Redshift streaming ingestion	a materialized view on Kinesis or MSK	streaming with **no S3 staging** (Lesson 23)
Zero-ETL integration	fully managed replication	source tables you want mirrored, no pipeline (Lesson 21)
Datashare with `--allow-writes`	a consumer writes to producer data	cross-account write scenarios (Lesson 16)
Amazon AppFlow	scheduled SaaS objects	SaaS sources you would rather not code

L13 · Everything That Touches Redshift

PATH	REACHES
Spectrum	S3 external tables, through the catalog
Federated query	live RDS / Aurora PostgreSQL or MySQL
Datashare	another Redshift namespace, live
Lambda UDF	calls a function part-way through a query

Why this matters

Three practical consequences:

1. **You rarely need to invent a path.** If you are writing custom code to get data in or out, check this list first — the odds are good that a managed path exists.
2. **Security surface is bigger than "the database password".** Eleven ways in means eleven things to govern, which is Lesson 17.
3. **The zero-copy paths change architecture.** Spectrum, federated query and datashares mean "get the data into Redshift" is a *choice*, not a prerequisite — which is Lesson 18.

Checklist

- ☐ I can name at least eight write paths without looking
- ☐ I can name at least six read paths
- ☐ I know Redshift can read S3, live databases and other warehouses
- ☐ I know `UNLOAD` exists and what it is for
- ☐ I know the Data API is the right choice from Lambda
- ☐ I have a photo or printout of this matrix

You've got it when you can…

…be asked "how do we get X into Redshift?" and answer with the *right* mechanism and the reason — including the cases where the answer is "we don't; we point at it instead."

MODULE 0 · LESSON 14

Getting Data In

Eleven write paths group into four families. Pick the family by latency first, then by how much transformation you need on the way in.

1 · WHAT IT IS

Four families, and they are not interchangeable

Each family trades freshness against control. The more managed the path, the less you get to change the data on the way through.

2 · HOW IT WORKS

FROM SLOWEST AND MOST CONTROLLED, TO FASTEST AND LEAST

BULK — COPY · hours

Files staged on S3, then COPY. Full transformation control. The default above a few million rows.

PROGRAMMATIC — Data API · JDBC

Applications, Lambda and orchestration issuing SQL. Correct for small volumes, wrong for large.

MANAGED REPLICATION — DMS · zero-ETL · minutes

Continuous, no code, source-shaped tables. You give up the chance to transform in flight.

STREAMING — Firehose · MV · seconds

Firehose stages through S3; streaming ingestion lands on a materialized view directly.

```
the more managed the path, the less you get to change the data in flight
```

3 · PICK BY LATENCY

Daily or hourly batch → Glue then COPY

Continuous replication → DMS or zero-ETL

Seconds → Redshift streaming ingestion

NEVER

An application writing rows one at a time

4 · IN PRACTICE

Glue writes Iceberg; dbt builds gold.

Redshift is loaded by dbt, not by jobs.

One writer role. Nothing else has INSERT.

one way in, on purpose

IN PLAIN ENGLISH

Freight, courier, direct line, or live feed. All four deliver — but you cannot repack the box once you have chosen the fast ones.

L14 · Getting Data In

Slide: `_render/L14-getting-data-in.html`

What it is

Eleven write paths group into **four families**, and they are not interchangeable. Each family trades freshness against control:

> **The more managed the path, the less you get to change the data in flight.**

That sentence is the whole lesson. Say it before choosing.

The four families

1. Bulk — hours

`COPY` from files staged on S3.

- Full transformation control: whatever wrote the files decided the shape
- Cheapest per row by a wide margin
- The default for anything above a few million rows
- Latency is however often you run it

2. Programmatic — seconds, small volumes

Redshift Data API, JDBC/ODBC, SQL issued by applications and orchestrators.

- Correct for small volumes: reference data, control-plane updates, a single corrected row
- Wrong for large volumes — see Lesson 11 on why row-by-row loading is fatal
- The **Data API** is the right choice from Lambda and Step Functions: no driver, no connection to hold open, no VPC plumbing

3. Managed replication — minutes to an hour

AWS DMS and zero-ETL integrations.

- Continuous, no code to write, no cluster to size
- Tables arrive **source-shaped** — you get what the source has, including its column names and its quirks
- **You give up the chance to transform on the way.** Any shaping happens after landing, inside the target

4. Streaming — seconds

Amazon Data Firehose and Redshift streaming ingestion.

- **Firehose** stages through S3 and then issues `COPY` — simple, no code, slight buffering delay
- **Streaming ingestion** maps a materialized view directly onto Kinesis or MSK with **no S3 staging at all** — lower latency, and the object you get is an MV (Lesson 12)

Picking by latency

REQUIREMENT	MECHANISM
Daily or hourly batch	Glue → S3 → `COPY`
Continuous replication	DMS or zero-ETL
Seconds	Redshift streaming ingestion
A handful of rows from an app	Data API

Never: an application writing rows one at a time into a warehouse. If an application needs to record events, it writes to a queue or a stream, and something else lands them in bulk.

L14 · Getting Data In

The trade to state out loud

Managed paths are genuinely attractive — no pipeline to build, nothing to page you at 3am. But they hand you the source's shape, and source shapes are rarely what a report wants.

The honest question is not "which is easiest?" but **"where does the transformation happen, and who owns it?"** With a managed path, the answer is "after landing, in the warehouse, in SQL." That may be fine. It is just a different place to put the work, not less work.

In practice

- Glue writes Iceberg; **dbt builds gold**. Redshift is loaded by dbt, not directly by ingestion jobs.
- **One writer role.** Nothing else in the account holds `INSERT` on gold tables.
- That is deliberate: one way in means one place to look when a number is wrong.

Checklist

- ☐ I can name the four families and their latencies
- ☐ I can state the freshness-versus-control trade in one sentence
- ☐ I know why the Data API beats JDBC from Lambda
- ☐ I know the difference between Firehose and streaming ingestion
- ☐ I would push back on an application writing rows directly
- ☐ I know which role is allowed to write our gold tables

You've got it when you can…

…be given a new source and a freshness requirement, pick the family in under a minute, and explain what transformation control you are giving up by choosing it.

MODULE 0 · LESSON 15

Getting Data Out Again

A warehouse that only lets people in through one door becomes the exact bottleneck it was built to remove.

1 · WHAT IT IS

Reading is not just BI — four different consumers need four different doors

Analysts, applications, other engines and machine learning all want the same numbers, and none of them wants them the same way.

2 · HOW IT WORKS

FOUR EXITS, AND THE FIRST ONE IS THE MOST OVERLOOKED

UNLOAD — `UNLOAD ... TO 's3://...'`
Writes results to S3 as Parquet. How gold data gets republished for cheaper engines to read.

JDBC / ODBC — Power BI · Tableau · QuickSight
The BI path. Point it at views, never at raw fact tables, so you can change the model underneath.

DATA API — `redshift-data`
HTTP calls, no driver and no held connection. The right choice from Lambda and Step Functions.

ENGINES & ML — Glue · EMR · SageMaker
Spark reads frames straight out; Redshift ML trains and scores without the data ever leaving.

```
publishing gold back to S3 stops Redshift from becoming the only way to the data
```

3 · PICK BY CONSUMER

Dashboards → JDBC onto views
Applications and Lambda → Data API
Other engines → UNLOAD to Parquet

NEVER
Let BI tools query the fact tables directly

4 · IN PRACTICE

Power BI reads reporting views only.
Views are late-binding, so models can move.
The BI role has SELECT and nothing else.

views are the contract

IN PLAIN ENGLISH

Give people the right exit and they stop climbing through the windows. Every unofficial copy of your data started as a missing door.

L15 · Getting Data Out Again

Slide: `_render/L15-getting-data-out.html`

What it is

A warehouse that only lets people in through one door becomes the exact bottleneck it was built to remove.

Four different kinds of consumer want the same numbers, and none of them wants them the same way. Give each the right exit and they stop building workarounds — because **every unofficial copy of your data started life as a missing door**.

The four exits

1. UNLOAD — the most overlooked

```
UNLOAD ('SELECT * FROM gold.fct_sales_line WHERE sale_date >= ...')
TO 's3://bucket/published/fct_sales_line/'
IAM_ROLE 'arn:aws:iam::...:role/redshift-unloader'
FORMAT AS PARQUET
PARTITION BY (sale_date);
```

Writes query results to S3 as Parquet. This is how **gold data gets republished back to the lake** so cheaper engines — Athena, Spark, a data scientist's notebook — can read it without touching the warehouse at all.

Publishing gold back to S3 is what stops Redshift from becoming the only way to the data. It is the single highest-leverage thing in this lesson.

2. JDBC / ODBC — the BI path

Power BI, Tableau and QuickSight all connect this way.

Point them at views, never at raw fact tables. A view is a contract: you can restructure the model underneath it, add a column, change a distribution key, and the report keeps working. Point a report at a base table and you have frozen your schema by accident.

Late-binding views are useful here: they are not bound to the underlying objects until query time, so you can rebuild the model beneath them without dropping and recreating the view.

3. Data API — for applications

HTTP calls, no driver, no persistent connection. The right choice from Lambda, Step Functions, and any application that would otherwise need VPC plumbing and a connection pool to talk to the warehouse.

4. Engines and ML

Glue, EMR and Spark read frames straight out through the Redshift connector. **Redshift ML** trains and scores without the data leaving the warehouse at all.

Picking by consumer

CONSUMER	EXIT
Dashboards	JDBC onto **views**
Applications and Lambda	Data API
Other engines, data science	`UNLOAD` to Parquet
ML training	Spark connector or Redshift ML

Never let BI tools query the fact tables directly. It looks like it works right up until you need to change the model.

L15 · Getting Data Out Again

In practice

- Power BI reads **reporting views only**.
- Views are **late-binding**, so models can be rebuilt beneath them.
- The BI role has `SELECT` on views and nothing else — no base tables, no write.

That last point is worth stating: the reader/writer split (Lesson 17) is enforced through which objects each role can even see.

Checklist

- ☐ I know `UNLOAD` exists and what publishing gold back to S3 buys
- ☐ I point BI at views, never base tables, and can say why
- ☐ I know why late-binding views help
- ☐ I use the Data API from Lambda rather than a JDBC driver
- ☐ I can name what the BI role is permitted to do
- ☐ I can spot an unofficial data copy and name the missing door that caused it

You've got it when you can…

…find a spreadsheet someone maintains by hand from a weekly CSV export, work out which exit was missing, and replace it — instead of just asking them to stop.

MODULE 0 · LESSON 16

Reading Without Copying

Three doors let Redshift see data it does not store — and one of them changed recently in a way most people have missed.

1 · WHAT IT IS

An external schema is a pointer, not a copy

The table appears in Redshift and you query it with ordinary SQL. The bytes never move — which means the cost moves, it does not vanish.

2 · HOW IT WORKS

THREE DOORS — POINTING AT THREE DIFFERENT KINDS OF THING

SPECTRUM → S3 — `CREATE EXTERNAL SCHEMA`

External tables over the lake. Reads Parquet, ORC, CSV, JSON, Avro — plus Iceberg, Hudi CoW and Delta.

FEDERATED → LIVE DB — `RDS / Aurora PostgreSQL · MySQL`

Queries the operational database as it is right now. Filters are pushed down to the source.

DATASHARE → REDSHIFT — `producer / consumer`

Live data from another cluster, account or Region. No copy, no ETL and no lag at all.

CORRECTION WORTH MAKING OUT LOUD — DATASHARES ARE NO LONGER READ-ONLY

authorize-data-share --allow-writes lets a consumer INSERT and UPDATE producer data.

```
pointing at data does not make it free — it moves the bill from storage to every query
```

3 · WHEN TO USE WHICH

Spectrum — cold history you rarely scan

Federated — small, live lookups

Datashare — another team's warehouse

NEVER

Federated query for a big analytical scan

4 · IN PRACTICE

An external schema exposes S3 Tables data.
Redshift reads Iceberg it never loaded.
Gold is loaded; silver is pointed at.

load what you join hard

IN PLAIN ENGLISH

A library card, not a photocopy. You can read the book any time — but you walk to the other building every single time you open it.

L16 · Reading Without Copying

Slide: `_render/L16-reading-without-copying.html`

What it is

An **external schema is a pointer, not a copy**. The table appears inside Redshift and you query it with ordinary SQL — but the bytes never move.

Which means: **the cost moves, it does not vanish.** You have swapped a storage bill and a load job for a per-query bill somewhere else. That is often a good trade. It is never a free one.

Three doors, pointing at three different kinds of thing.

1. Spectrum → S3

```
CREATE EXTERNAL SCHEMA lake_ext
FROM DATA CATALOG
DATABASE 'silver'
IAM_ROLE 'arn:aws:iam::...:role/redshift-spectrum';
```

Redshift reads files in the lake through the Glue Data Catalog, without loading them.

Formats it reads: Parquet, ORC, CSV, JSON, Avro — plus the transactional table formats: **Apache Iceberg**, **Apache Hudi Copy-on-Write**, and **Delta Lake** (via symlink manifest tables).

You can also build a **materialized view over external tables** with incremental maintenance — caching an expensive S3 scan inside the warehouse (Lesson 12).

2. Federated query → a live database

```
CREATE EXTERNAL SCHEMA live_pg
FROM POSTGRES
DATABASE 'orders' SCHEMA 'public'
URI 'my-aurora-cluster...rds.amazonaws.com'
IAM_ROLE '...' SECRET_ARN '...';
```

Redshift queries an **RDS or Aurora PostgreSQL or MySQL** database *as it is right now*. Filters are **pushed down** to the source, so the remote database does the narrowing rather than shipping everything back.

The catch is the obvious one: this puts analytical load on a production transactional database. Predicate pushdown makes small lookups cheap; it does not make a large scan safe.

3. Datashares → another Redshift

Live data from another cluster, account or Region. No copy, no ETL, no lag. The producer creates a datashare, adds objects, and authorises a consumer; the consumer creates a database from it and queries it like any other.

The correction worth making out loud

Datashares are no longer read-only.

```
aws redshift authorize-data-share \
  --data-share-arn arn:aws:redshift:...:datashare:.../salesshare \
  --consumer-identifier <consumer> \
  --allow-writes
```

With `--allow-writes`, a consumer can `INSERT` and `UPDATE` the producer's data. Most material written before this landed says datashares are read-only. It is worth knowing, and worth being careful with — cross-account write access is a serious grant.

L16 · Reading Without Copying

When to use which

DOOR	RIGHT FOR
Spectrum	cold history you scan rarely
Federated query	small, live lookups
Datashare	another team's warehouse

Never use federated query for a big analytical scan. You will find out about it from the on-call engineer of the production system, not from your own monitoring.

In practice

- An external schema exposes S3 Tables data inside Redshift, so Redshift reads Iceberg it never loaded.
- **Gold is loaded; silver is pointed at.** That split is deliberate.
- The rule of thumb: **load what you join hard, point at the rest.**

Checklist

- ☐ I can write a `CREATE EXTERNAL SCHEMA` for both Spectrum and federated
- ☐ I know which file and table formats Spectrum reads
- ☐ I can explain predicate pushdown and its limits
- ☐ I know datashares support writes, and what flag enables it
- ☐ I would refuse a federated query used for a large scan
- ☐ I know which of our layers is loaded and which is pointed at

You've got it when you can…

…be asked to "just point Redshift at the production database" for a daily report, explain exactly what that would do to the production database, and offer the right alternative instead.

MODULE 0 · LESSON 17

Who Is Actually Allowed

Two permission systems on one platform. Redshift governs its own tables; Lake Formation governs everything read through the catalog.

1 · WHAT IT IS

If two systems can grant access to the same table, you must know which one wins

This is the single most common source of “it works for me but not for them” on a lakehouse. Learn where the boundary sits.

2 · HOW IT WORKS

FOUR LAYERS, FROM THE DATABASE OUTWARDS

REDSHIFT ROLES — `GRANT ... TO ROLE`

Users, groups and roles. GRANT on schemas and tables, plus row-level and column-level security.

IAM AUTHENTICATION — `IAM · Identity Center`

Federated login instead of database passwords. The identity decides what the session may do.

LAKE FORMATION — `catalog · database · table · column`

Grant once, and every engine reading through the catalog enforces it — Athena, Spark, Redshift.

READER / WRITER SPLIT — `the pattern that matters`

The ETL role writes. The BI role reads reporting views. No human account holds both.

```
Redshift-native tables -> GRANT   ·   anything via the catalog -> Lake Formation
```

3 · RULES OF THUMB

Grant to roles, never to individual users

Mask PII at the column, not in the report

Expose views to BI, never base tables

NEVER

Share one superuser between people or jobs

4 · IN PRACTICE

Epsilon customer data is PII — mask it.
Column-level grants, not a filtered copy.
Every role is Terraform, never console.

access is code, and it is reviewed

IN PLAIN ENGLISH

Two sets of keys to one building: the office door and the filing cabinet. Knowing which lock you are holding is most of the job.

L17 · Who Is Actually Allowed

Slide: `_render/L17-permissions.html`

What it is

Two permission systems govern one platform:

- **Redshift** governs its own tables.
- **Lake Formation** governs everything read **through the catalog**.

> If two systems can grant access to the same table, you must know which one wins.

That sentence is the single most common source of "it works for me but not for them" on a lakehouse. Learn where the boundary sits before you debug it at speed.

Four layers, from the database outwards

1. Redshift roles

Users, groups and **roles**. `GRANT` on schemas and tables, plus:

- **Row-level security (RLS)** — a policy limits which rows a role can see
- **Column-level security (CLS)** — `GRANT SELECT (col_a, col_b)` rather than on the whole table

```
CREATE ROLE bi_reader;
GRANT USAGE ON SCHEMA reporting TO ROLE bi_reader;
GRANT SELECT ON ALL TABLES IN SCHEMA reporting TO ROLE bi_reader;
```

Grant to **roles**, never to individual users. A user who leaves should be removed from a role, not audited for scattered grants.

2. IAM authentication

Federated login through IAM or IAM Identity Center instead of database passwords. The identity decides what the session may do, and there is no shared secret to rotate or leak.

3. Lake Formation

Grants at **catalog** → **database** → **table** → **column** level, and — this is the important part — **enforced by every engine that reads through the catalog**. Athena, Spark, EMR and Redshift Spectrum all honour the same grant.

Define the permission once; it applies everywhere. That is the whole value proposition, and it is why the catalog matters as much as the storage.

4. The reader/writer split

The pattern that matters more than any individual grant:

- The **ETL role** writes. It is assumed by jobs, never by people.
- The **BI role** reads reporting views. Not base tables, not writes.
- **No human account holds both.**

Where the boundary sits

OBJECT	GOVERNED BY
Redshift-native tables and views	Redshift `GRANT`
External tables read via the catalog	**Lake Formation**
S3 Tables / Iceberg via the catalog	**Lake Formation**
Datashares	Redshift, plus Lake Formation for LF-managed shares

When someone can query a table in Athena but not through Spectrum — or the reverse — this table is where you start.

L17 · Who Is Actually Allowed

Rules of thumb

- Grant to **roles**, never to individual users
- Mask PII at the **column**, not in the report — a report-level filter is not a control
- Expose **views** to BI, never base tables
- **Never** share one superuser between people or between jobs

In practice

- Epsilon customer data is **PII**. It is masked at the column level, with grants — not by maintaining a separate filtered copy, which would be a second thing to keep correct.
- Every role is defined in **Terraform**, never clicked in the console. Access is code, and it is reviewed like code.

That last point is not bureaucracy. An access grant made in the console has no author, no reason and no review — and nobody will ever dare remove it.

Checklist

- ☐ I know which system governs which kind of object
- ☐ I grant to roles, never to users
- ☐ I can write a column-level grant
- ☐ I know what RLS and CLS are and when to reach for each
- ☐ I can explain the reader/writer split and why no human holds both
- ☐ I know that our roles live in Terraform, and why that matters

You've got it when you can…

…debug "she can see it in Athena but not in Redshift" by asking which system owns that object — and fix it in the right place rather than by adding another grant somewhere and hoping.

MODULE 0 · LESSON 18

Copy It, Or Point At It?

One question, four answers, four different bills. Decide this per table — not once for the whole platform.

1 · THE QUESTION

Does this table earn its storage — or is it cheaper to fetch it when someone asks?

Copying costs storage once and query time never. Pointing costs storage never and query time every single time.

2 · THE SAME TABLE, FOUR WAYS

LOAD IT

Data lives inside Redshift
Fastest joins, full SQL, sort keys
You store the bytes twice
As fresh as the last load

YOU PAY FOR
`storage + the load job`

GOLD FACTS & DIMS

POINT AT S3

Data stays in the lake
No second copy, no load job
Slower joins; you scan every time
As fresh as whatever wrote the files

YOU PAY FOR
`bytes scanned per query`

COLD HISTORY

POINT AT THE OLTP DB

Data stays in the live database
Always current — zero lag
Puts analytical load on production
Filters push down; big scans do not

YOU PAY FOR
`load on the source system`

SMALL LIVE LOOKUPS

SHARE IT

Data stays in another warehouse
Live, no copy, across accounts
Both sides must be Redshift
Writes possible with allow-writes

YOU PAY FOR
`the consumer's compute`

ANOTHER TEAM'S DATA

Copy what you join hard and often. Point at what you scan rarely. Never point at production for a report that runs every hour.

IN PLAIN ENGLISH

Buy the book you read weekly. Borrow the one you open twice a year. Do not borrow it from a colleague who is still using it.

L18 · Copy It, Or Point At It?

Slide: `_render/L18-copy-or-point.html`

The question

> Does this table earn its storage — or is it cheaper to fetch it when someone asks?

Copying costs storage **once** and query time **never**. Pointing costs storage **never** and query time **every single time**.

Neither is free. The mistake is treating "zero-copy" as if it meant "zero-cost".

The same table, four ways

Load it — `COPY` + `MERGE`

- Data lives inside Redshift
- Fastest joins, full SQL, sort keys and distribution work for you
- You store the bytes twice
- As fresh as the last load
- **You pay for:** storage plus the load job
- **Use for:** gold facts and dimensions

Point at S3 — Spectrum

- Data stays in the lake
- No second copy and no load job to operate
- Slower joins; you scan the files on every query
- As fresh as whatever wrote the files
- **You pay for:** bytes scanned, per query
- **Use for:** cold history

Point at the OLTP database — federated query

- Data stays in the live database
- Always current — zero lag
- **Puts analytical load on production**
- Filters push down; large scans do not
- **You pay for:** load on the source system
- **Use for:** small live lookups

Share it — datashare

- Data stays in another warehouse
- Live, no copy, works across accounts and Regions
- Both sides must be Redshift
- Writes are possible with `--allow-writes`
- **You pay for:** the consumer's own compute
- **Use for:** another team's data

The rule

> **Copy what you join hard and often. Point at what you scan rarely. Never point at production for a report that runs every hour.**

How to actually decide

Four questions, in order:

L18 · Copy It, Or Point At It?

1. **How often is it queried?** Many times a day → lean toward loading. Twice a month → point at it.
2. **Is it joined, or just filtered?** Heavy joins want co-located data and sort keys. A filtered scan does not.
3. **How fresh must it be?** "Right now" narrows you to federated query or a datashare. "This morning" opens everything.
4. **Who pays, and do they know?** A pointer moves cost onto the querier, or onto someone else's production system. That is a decision to make explicitly, not to discover.

The trap

Zero-copy architectures demo beautifully and degrade quietly. The failure looks like this: someone points at a source for a small lookup, the report gets popular, the query runs hourly instead of weekly, and six months later a production database is struggling for a reason nobody connects to the dashboard.

The defence is to write down, per external schema, **what it is for and roughly how often it should be hit** — and to alarm when reality diverges.

In practice

- **Gold is loaded. Silver is pointed at.**
- That is not an accident of implementation; it follows directly from the rule above. Gold is joined hard and queried constantly. Silver is scanned occasionally for reprocessing and investigation.

Checklist

- ☐ I can state the trade in one sentence each way
- ☐ I know the four options and what each costs
- ☐ I can name who pays for each
- ☐ I decide this per table, not once for the platform
- ☐ I would never point a frequent report at a production database
- ☐ I know why gold is loaded and silver is not

You've got it when you can…

…design a new dataset's access path in a design review, name who pays for it, and predict what would go wrong if the query frequency went up tenfold.

MODULE 0 · LESSON 19

Eight Kinds Of Source

Classify the source before you choose the pipeline. The class decides the mechanism — and every project has more classes than it expects.

1 · THE TAXONOMY

KIND OF SOURCE	WHAT MAKES IT HARD	HOW YOU MOVE IT	APPAREL GROUP
OLTP RELATIONAL	Huge tables, live load, no downtime window	JDBC · CDC · zero-ETL	`RMS · SIM · XStore`
SAAS API	Rate limits, cursor paging, no bulk export	API pull · AppFlow · zero-ETL	`Epsilon · MoEngage`
E-COMMERCE	Sometimes a database, sometimes an API, sometimes both	JDBC or API	`Magento`
FILE DROP	Arrives late, arrives twice, or does not arrive	S3 landing · Transfer Family	`Vemco · Irisys`
STREAMS & EVENTS	Unbounded; ordering and duplicates both matter	Kinesis · MSK · Firehose	`none today`
LOGS & CLICKSTREAM	Enormous volume, very low value per single row	Firehose to S3, partitioned	`web analytics`
ON-PREM	Behind a firewall; needs a network path before anything	VPN / Direct Connect + Glue	`legacy stores`
THIRD-PARTY FEED	You do not control the schema, and it will change	contract + validate on landing	`market data`

```
eight sources, five classes — the variety is in the connectors, not in a hundred bespoke jobs
```

IN PLAIN ENGLISH

You do not need a different van for every delivery. You need to know which of five kinds of parcel you are being handed.

L19 · Eight Kinds Of Source

Slide: `_render/L19-source-taxonomy.html`

What it is

Classify the source **before** you choose the pipeline. The class decides the mechanism — and every project turns out to have more classes than the kick-off deck admitted.

The payoff is direct: eight sources reduce to about five classes, and five classes reduce to three connectors. The variety lives in the configuration, not in a hundred bespoke jobs.

The taxonomy

KIND	WHAT MAKES IT HARD	HOW YOU MOVE IT	APPAREL GROUP
OLTP relational	huge tables, live load, no downtime window	JDBC batch · CDC · zero-ETL	RMS · SIM · XStore
SaaS API	rate limits, cursor paging, no bulk export	API pull · AppFlow · zero-ETL	Epsilon · MoEngage
E-commerce	sometimes a DB, sometimes an API, sometimes both	JDBC or API	Magento
File drop	arrives late, arrives twice, or not at all	S3 landing · Transfer Family	Vemco · Irisys
Streams & events	unbounded; ordering and duplicates both matter	Kinesis · MSK · Firehose	none today
Logs & clickstream	enormous volume, very low value per row	Firehose to S3, partitioned	web analytics
On-prem	behind a firewall; needs a network path first	VPN / Direct Connect + Glue	legacy stores
Third-party feed	you do not control the schema, and it will change	contract + validate on landing	market data

What each class demands of you

OLTP relational

The question that decides everything: **does the source hard-delete rows?** If it does, a timestamp-based batch pull will silently drift wrong and never tell you — see Lesson 22. Ask this on day one, of every relational source.

Also ask: is there a reliable modification timestamp, is it indexed, and is it set by the database or by the application? Application-set timestamps lie.

SaaS API

Rate limits and paging are the whole engineering problem. Assume the API will be slow, will page inconsistently, and will change without notice. Budget for full reloads being impossible.

File drop

The three failure modes are in the table: late, twice, and never. All three are normal. Design for them rather than treating each as an incident — idempotent loads make "twice" harmless and a freshness alarm makes "never" visible.

On-prem

The network path is a **prerequisite, not a task**. It is procured, not coded, and it has a lead time measured in weeks. Raise it in week one or it becomes the critical path in week nine.

L19 · Eight Kinds Of Source

Third-party feed

You do not control the schema. So write down the contract you expect, validate against it on landing, and fail loudly when reality diverges — rather than discovering it downstream in a report.

In practice

Eight Apparel Group sources, five classes, **three connectors**:

- Three Oracle databases → **one Oracle JDBC connector**
- Two cursor-paged SaaS APIs → **one API connector**
- Two footfall file feeds → **one file-drop connector**
- Magento → whichever of the first two it turns out to be

Checklist

- ☐ I can classify a new source in under a minute
- ☐ I ask about hard deletes on every relational source
- ☐ I ask about rate limits and paging on every API source
- ☐ I raise network access for on-prem sources in week one
- ☐ I know how many connector classes our eight sources actually need
- ☐ I write a contract for feeds whose schema I do not control

You've got it when you can…

…be handed a list of source systems you have never seen, group them into classes, and estimate the work as "N connectors plus one spec per table" — instead of "N pipelines".

MODULE 0 · LESSON 20

Six Ways To Move Data

The table you will argue from in every design review. Latency improves down the list; the control you keep gets worse.

1 · THE DECISION TABLE

MECHANISM	AWS SERVICE	LATENCY	CHOOSE IT WHEN
BATCH ETL	AWS Glue · EMR (Spark)	`hours`	You need to transform on the way and control the shape the default for a medallion lakehouse
CDC REPLICATION	AWS DMS	`minutes`	Continuous replication between two different engines reads the source's log — no load on its tables
ZERO-ETL	managed integration	`min – 1 hr`	You want the table in the target with no pipeline at all Oracle, SAP, Salesforce, DynamoDB and more — Lesson 21
FEDERATED QUERY	Redshift · Athena	`live`	A small lookup against a live database, with no copy filters push down; large scans do not
STREAMING	Kinesis · MSK · Firehose	`seconds`	Genuinely event-driven data, where seconds change a decision most retail reporting does not need this
FILE TRANSFER	Transfer Family · DataSync · AppFlow	`scheduled`	SFTP drops, NAS sync and SaaS connectors you would rather not code the boring one that quietly works

```
pick the latency you actually need, then the most managed option that still gives you it
```

IN PLAIN ENGLISH

Post, courier, direct debit, phone call, live feed, or the weekly van. Same parcel — six prices, six speeds, six amounts of fuss.

L20 · Six Ways To Move Data

Slide: `_render/L20-six-ways-to-move-data.html`

The decision table

MECHANISM	AWS SERVICE	LATENCY	CHOOSE IT WHEN
Batch ETL	AWS Glue · EMR (Spark)	hours	you need to transform on the way and control the shape
CDC replication	AWS DMS	minutes	continuous replication between two different engines
Zero-ETL	managed integration	minutes – 1 hr	you want the table in the target with **no pipeline at all**
Federated query	Redshift · Athena	live	a small lookup against a live database, with no copy
Streaming	Kinesis · MSK · Firehose	seconds	genuinely event-driven data, where seconds change a decision
File transfer	Transfer Family · DataSync · AppFlow	scheduled	SFTP drops, NAS sync and SaaS connectors you would rather not code

The rule

> **Pick the latency you actually need, then take the most managed option that still gives you it.**

Latency improves down the list. The control you keep gets worse. Those two facts move together and there is no configuration that separates them.

Notes on each

Batch ETL — the default for a medallion lakehouse

You own the code, so you own the shape. This is why bronze and silver can be conformed, deduplicated and typed rather than mirroring source quirks. Cheapest per row at volume.

CDC replication — reads the log, not the tables

DMS reads the source's redo/WAL/binlog, so it adds almost no query load to the source. It catches hard deletes, which a watermark pull cannot. Lesson 22.

Zero-ETL — far broader than most people know

Oracle, SQL Server, MySQL, PostgreSQL, DynamoDB, SAP OData, Salesforce, ServiceNow and more. Targets include Redshift, S3, S3 Tables and RMS. Lesson 21 has the full matrix and the limits — read it before you rule this out.

Federated query — live, but the source pays

Filters push down. Large scans do not. Fine for a lookup, dangerous for a report. Lesson 16.

Streaming — most retail reporting does not need this

Kept in the course so you can recognise the minority of cases that genuinely do. Lesson 23.

File transfer — the boring one that quietly works

Transfer Family for SFTP, DataSync for filesystem sync, AppFlow for SaaS objects. No cluster, no code, and it will still be running unattended in three years.

How to use this in a design review

Work down the columns in this order:

L20 · Six Ways To Move Data

1. **What latency does the decision actually need?** Not what someone asked for — what the decision being made needs. "Real-time" usually means "not yesterday's".
2. **Does the shape need to change on the way?** If yes, managed replication is out for that source.
3. **Who operates it at 3am?** A path with no code has no code to debug, and that has real value.
4. **What does it cost when volume triples?** Per-row costs scale differently from always-on costs.

In practice

For the Apparel Group build, the interesting comparison is **batch ETL versus zero-ETL for the three Oracle sources**. That is a real decision with real arguments on both sides, and Lesson 21 sets it out properly.

Checklist

- ☐ I can reproduce the six-row table from memory
- ☐ I can state the latency-versus-control rule in one sentence
- ☐ I know which mechanism reads the source's log rather than its tables
- ☐ I know zero-ETL supports Oracle and SAP, not just Aurora
- ☐ I ask "what latency does the decision need" before "what latency was requested"
- ☐ I consider who operates it at 3am

You've got it when you can...

...sit in a design review where someone has already picked a mechanism, ask the four questions above, and either confirm the choice with reasons or change it — in under ten minutes.

MODULE 0 · LESSON 21

Zero-ETL: No Pipeline At All

A fully managed integration replicates the data and the schemas into the target, continuously. It covers far more sources than most people know.

1 · WHAT IT IS

There is no job, no cluster, no code and nothing to schedule

You configure an integration between a source and a target. AWS keeps them in step. That is the entire operational surface.

2 · SOURCES — THREE FAMILIES

AWS DATABASES

Aurora MySQL · Aurora PostgreSQL · RDS MySQL
Amazon DynamoDB · Oracle Database@AWS

SELF-MANAGED DATABASES

via AWS DMS

Oracle · SQL Server · MySQL · PostgreSQL
your own instances, wherever they run

SAAS APPLICATIONS

Salesforce · SAP OData · ServiceNow · Zendesk
Zoho CRM · Facebook Ads · Instagram Ads

WHY THIS MATTERS TO US

Three of Apparel Group's eight sources are Oracle. This is a real alternative to hand-built JDBC.

3 · TARGETS

- Amazon Redshift data warehouse
- Amazon S3 general-purpose bucket
- Amazon S3 Tables
- Redshift Managed Storage

the last three go via SageMaker Lakehouse

4 · LIMITS THAT DECIDE THE DESIGN

Self-managed sources → Redshift only
SaaS sources: 1 hour minimum latency
It replicates. It does not transform.

check the limits before you commit

IN PLAIN ENGLISH

A pipe someone else maintains. You lose the right to change the water on the way through — and you stop being paged at 3am.

L21 · Zero-ETL: No Pipeline At All

Slide: `_render/L21-zero-etl.html`

What it is

A **fully managed integration** that replicates transactional data **and schemas** from a source into a target, continuously and in near real time.

There is no job, no cluster, no code and nothing to schedule. You configure an integration between a source and a target, and AWS keeps them in step. That is the entire operational surface.

The reason this lesson matters is that most people's mental model of zero-ETL stopped at "Aurora to Redshift". It is now far broader than that.

Sources — three families

AWS databases

- Aurora MySQL
- Aurora PostgreSQL
- RDS MySQL
- **Amazon DynamoDB**
- **Oracle Database@AWS (ODB)**

Self-managed databases — via AWS DMS

- **Oracle**
- SQL Server
- MySQL
- PostgreSQL

Your own instances, wherever they run.

SaaS applications

- Salesforce
- Salesforce Marketing Cloud Account Engagement
- **SAP OData**
- ServiceNow
- Zendesk
- Zoho CRM
- Facebook Ads · Instagram Ads

Targets

- **Amazon Redshift** data warehouse
- **Amazon S3** general-purpose bucket
- **Amazon S3 Tables**
- **Redshift Managed Storage**

The last three go via the **SageMaker Lakehouse architecture** — which means zero-ETL is not only a warehouse-loading tool. It can land into the lake.

The limits that decide the design

Three of them, and each one can rule the option out:

L21 · Zero-ETL: No Pipeline At All

1. **Self-managed database sources can replicate only to a Redshift data warehouse.** Not to S3, S3 Tables or RMS. If your architecture lands everything in the lake first, this is a genuine constraint.
2. **Application (SaaS) sources have a minimum latency of one hour.** If someone specified "near real-time" for a Salesforce feed, this is the number to put in front of them.
3. **It replicates. It does not transform.** Tables arrive source-shaped, with the source's column names, types and quirks. All shaping happens after landing.

Why this matters to us

Three of Apparel Group's eight sources are Oracle, and self-managed Oracle → Redshift is now supported.

That makes zero-ETL a *credible alternative* to hand-building JDBC pipelines for those three sources, and the team should be able to argue it honestly rather than defaulting either way.

Arguments for zero-ETL here: - No pipeline to build, test, deploy or operate for three large sources - No watermark logic, and therefore no watermark bugs - Near-real-time instead of daily, at no extra engineering cost

Arguments for keeping Glue: - **Medallion layering.** Zero-ETL puts source-shaped tables in Redshift; our architecture wants typed, conformed Iceberg in the lake first. And self-managed sources cannot target S3 Tables. - **Transformation control.** Business rules — the XStore receipt logic, VAT splitting, pack-size parsing — have to run somewhere. In a zero-ETL design they run in Redshift SQL instead of Spark, which is a different skill set and a different cost profile. - **Cost at volume.** XStore is the giant. Continuous replication of a very large, high-churn table has an ongoing cost that a nightly batch does not. - **One engine to operate.** A platform that is mostly Glue plus three zero-ETL integrations has two operational models, two monitoring stories and two failure vocabularies.

The honest conclusion: it is worth a spike on one Oracle source before committing either way. Do not decide it in a slide.

Checklist

- ☐ I can name all three source families and give examples of each
- ☐ I know all four targets
- ☐ I know self-managed sources can only target Redshift
- ☐ I know SaaS sources have a one-hour minimum latency
- ☐ I know it replicates but does not transform
- ☐ I can argue both sides of zero-ETL versus Glue for an Oracle source

You've got it when you can…

…be told "we'll build a Glue pipeline for Oracle" and raise zero-ETL as a real option with its actual limits attached — then help the team decide on evidence rather than on habit.

MODULE 0 · LESSON 22

Change Data Capture

Reading the database's own transaction log instead of asking its tables what changed. That difference is the whole point.

1 · WHAT IT IS

A watermark pull asks "what changed since?" — CDC is told, by the database itself

Every relational database already writes a log of every change so it can recover. CDC reads that log instead of querying your tables.

2 · HOW IT WORKS

FOUR THINGS TO UNDERSTAND BEFORE YOU CHOOSE IT

READ THE LOG — `redo · WAL · binlog`

DMS reads the engine's change log, so it adds almost no query load to the source system.

FULL LOAD, THEN CDC — `one task, two phases`

One task copies the table, notes the log position, then streams every change from there on.

IT CATCHES DELETES — `the usual deciding factor`

Hard deletes and out-of-order updates that a timestamp-based pull silently misses forever.

SCHEMA DRIFT — `decide the policy first`

A new column upstream should be a decision you make, not a surprise you discover at 6am.

```
if the source hard-deletes rows, a watermark pull will be wrong and never tell you
```

3 · WHEN TO USE IT

Continuous replication at low latency

Sources that hard-delete rows

A different engine at each end

NOT FOR

When you must transform data on the way

4 · IN PRACTICE

Module 2 teaches watermarked batch pulls.
CDC is the alternative. Know both well.
Deletes usually force the decision.

ask about deletes on day one

IN PLAIN ENGLISH

Do not walk the shelves every night counting what moved. Read the till roll — it already lists every single thing that happened.

L22 · Change Data Capture

Slide: `_render/L22-cdc-and-dms.html`

What it is

> A watermark pull asks *"what changed since?"* — CDC is **told**, by the database itself.

Every relational database already writes a log of every change so it can recover from a crash. CDC reads that log instead of querying your tables.

On AWS: AWS DMS, with Redshift, S3 and others as targets.

How it works

1. Read the log

DMS reads the engine's change log — Oracle redo, PostgreSQL WAL, MySQL binlog — so it adds almost no query load to the source system. That alone is often the deciding factor when the source is a production system with no capacity to spare.

2. Full load, then CDC

One task, two phases. It copies the table, notes the log position at which the copy was consistent, and then streams every change from that position onward. You do not have to coordinate the handover yourself.

3. It catches deletes

This is usually what decides the choice.

A timestamp-based watermark pull asks the source for rows where `modified_at > last_run`. A row that was *deleted* does not match that query — it is simply absent, and absence is indistinguishable from "nothing changed". The row stays in your warehouse forever.

CDC sees the delete in the log, because the log records it.

The same applies to out-of-order updates, and to any update that does not touch the modification timestamp — which happens more often than people expect when the timestamp is set by the application rather than by the database.

4. Schema drift

A new column appears upstream. What should happen?

That should be **a decision you made in advance**, not a surprise at 6am. Options are: ignore unknown columns, add them automatically, or fail the task loudly. All three are defensible; having no answer is not.

When to use it

Use CDC when: - You need continuous replication at low latency - The source hard-deletes rows - The engines at each end are different

Do not use it when: - You must transform the data on the way. CDC replicates; shaping happens after landing.

Against the alternative

Module 2 teaches **watermarked batch pulls** in depth — reading a modification column and tracking the high-water mark. That approach is simpler, cheaper, and entirely correct *when the source does not hard-delete and the watermark column is trustworthy.*

Know both, and know the question that separates them:

> **"Does this source ever hard-delete a row?"**

L22 · Change Data Capture

Ask it of every relational source, on day one, of someone who actually knows. "I don't think so" is not an answer; check the schema for a soft-delete flag, or check the log.

In practice

- Module 2's engine is built around watermarked pulls.
- CDC is the alternative you should be able to argue for when the source demands it.
- **Deletes usually force the decision.** Everything else is a preference; deletes are a correctness issue.

Checklist

- ☐ I can explain what a transaction log is and why CDC reads it
- ☐ I can explain full-load-then-CDC as one task in two phases
- ☐ I can explain precisely why a watermark pull misses hard deletes
- ☐ I ask about hard deletes on every relational source
- ☐ I have a schema-drift policy decided before the drift happens
- ☐ I can argue CDC versus watermark pull on the merits

You've got it when you can…

…be shown a warehouse table with more rows than the source, work out in one question that the source hard-deletes, and explain why the nightly pull will never fix it on its own.

MODULE 0 · LESSON 23

Streaming, Honestly

Six services, one materialized view — and a straight answer about whether your project needs any of it.

1 · WHAT IT IS

Two separate questions: how the events travel, and where they land

People mix these up constantly. Kinesis and Kafka are transport. Firehose and streaming ingestion are landing. You choose both.

2 · HOW IT WORKS

TRANSPORT — HOW EVENTS TRAVEL

KINESIS DATA STREAMS — shards · ordered
Durable ordered shards. Several consumers can read the same stream independently.

AMAZON MSK / KAFKA — incl. Confluent Cloud
Managed Kafka. The right call when the organisation already speaks Kafka.

LANDING — WHERE THEY COME TO REST

AMAZON DATA FIREHOSE — no code at all
Delivers to S3, Iceberg tables including S3 Tables, Redshift and OpenSearch.

REDSHIFT STREAMING INGESTION — AUTO REFRESH
A materialized view mapped straight onto the stream — with no S3 staging at all.

```
Firehose into Iceberg trades throughput against how many partitions are open at once
```

3 · WHEN TO USE IT

Seconds genuinely change a decision
The source emits events, not table rows
Volume is too high to batch sensibly

NOT FOR

Daily retail reporting — batch is cheaper

4 · IN PRACTICE

Neither of our platforms streams today.
Learn it to recognise the 10% that needs it.
Do not stream because it sounds modern.

batch until it genuinely hurts

IN PLAIN ENGLISH

A live feed is only worth its cost if somebody is watching the screen. Nobody watches yesterday's sales report at three in the morning.

L23 · Streaming, Honestly

Slide: `_render/L23-streaming.html`

What it is

Two separate questions that people mix up constantly:

1. **How do the events travel?** — Kinesis, MSK/Kafka
2. **Where do they come to rest?** — Firehose, Redshift streaming ingestion

You choose both. "We'll use Kinesis" answers only half the design.

Transport — how events travel

Amazon Kinesis Data Streams

Durable, ordered **shards**. Several consumers can read the same stream independently, each at its own position. Ordering is guaranteed within a shard, not across shards — which is why the partition key matters.

Amazon MSK / Apache Kafka

Managed Kafka, including Confluent Cloud as a source. The right call when the organisation already speaks Kafka — the tooling, the mental model and the people are already there.

Landing — where they come to rest

Amazon Data Firehose

Managed delivery with **no code at all**. Destinations include:

- **Amazon S3**
- **Apache Iceberg tables** — self-managed *or* **S3 Tables** — with insert, update and delete **routing** from a single stream
- **Amazon Redshift** — stages to S3, then issues `COPY`
- OpenSearch, HTTP endpoints and others

The trade-off worth teaching: **ingest throughput versus how many Iceberg partitions are open at once**. A stream fanning out across many partitions delivers less throughput than one writing to a few. That is a design constraint on your partitioning, not a tuning knob.

Redshift streaming ingestion

A **materialized view mapped directly onto the stream** — Kinesis, MSK, Apache Kafka or Confluent Cloud — refreshed from the stream **with no S3 staging at all**, with `AUTO REFRESH`.

This is the same object from Lesson 12, doing a new job. Lower latency than Firehose, and one fewer thing in the path.

When you need windows and state

Managed Service for Apache Flink or **Glue streaming ETL** — for aggregations over time windows, joins between streams, and anything that has to remember what it saw.

Event plumbing that is not really streaming

DynamoDB Streams, **S3 Event Notifications** and **EventBridge** trigger pipelines when something happens. They are event-driven, but they are not high-volume streaming, and reaching for Kinesis when an S3 event would do is a common over-build.

L23 · Streaming, Honestly

When to use it

Use streaming when: - Seconds genuinely change a decision someone is making - The source emits events, not rows in a table - Volume is too high to batch sensibly

Do not use it for: - Daily retail reporting. Batch is cheaper, simpler, and easier to reason about at 3am.

The honest position

Most retail analytics does not need streaming. This lesson exists so you can recognise the minority of cases that genuinely do — and so you are not talked into it by a diagram.

Two questions that end the conversation quickly:

1. **Who is watching the screen at 3am?** Freshness that nobody consumes is cost with no benefit.
2. **What decision changes in the next sixty seconds?** If none, you want a shorter batch interval, not a stream.

In practice

- Neither of our platforms streams today.
- Learn it to recognise the 10% that needs it.
- **Batch until it genuinely hurts.** Then measure what hurts, and stream only that.

Checklist

- ☐ I can separate transport from landing and name options for each
- ☐ I know Firehose writes to Iceberg including S3 Tables
- ☐ I know the throughput-versus-open-partitions trade
- ☐ I know streaming ingestion lands on an MV with no S3 staging
- ☐ I know when to reach for Flink instead
- ☐ I ask the two questions before agreeing to build a stream

You've got it when you can...

...hear "we need real-time" in a requirements session, ask what decision changes in the next minute, and land on the honest answer — which is usually an hourly batch.

MODULE 0 · LESSON 24

Eight Sources, One Platform

The hard part is not any single pipeline. It is making eight of them fail independently and land in the same shape.

1 · WHAT IT IS

Eight pipelines is not eight times one pipeline

One source is an engineering problem. Eight sources is a convention problem — and conventions have to be decided before the second one.

2 · HOW IT WORKS

FOUR CONVENTIONS THAT MAKE THE NINTH SOURCE CHEAP

ONE LANDING SHAPE — `raw/<source>/<table>/dt=...`
Every source lands under the same prefix pattern, partitioned the same way. No exceptions.

ONE CONTRACT PER SOURCE — `declared in config`
Schema, key, load mode and owner declared once — in configuration, never buried in code.

ISOLATION — `separate runs and state`
One bad feed must not stall the other seven. Separate runs, separate state, separate alarms.

LATE AND DUPLICATE ROWS — `assume both`
Rows will arrive late and twice. An idempotent MERGE on a key makes both harmless.

```
if adding the ninth source needs new code, the eighth was onboarded wrong
```

3 · DECIDE THESE EARLY

Partition key and its format
Timezone — pick one and write it down
What “yesterday” means for each source

NEVER
Let one source invent its own conventions

4 · IN PRACTICE

Three Oracle sources share one connector.
Two SaaS APIs share a second.
Footfall files share a third.

8 sources · 3 connectors

IN PLAIN ENGLISH

Every supplier delivers to the same bay, in the same boxes, with the same label. Otherwise the warehouse staff become the bottleneck.

L24 · Eight Sources, One Platform

Slide: `_render/L24-many-sources-one-platform.html`

What it is

Eight pipelines is not eight times one pipeline.

One source is an engineering problem. Eight sources is a *convention* problem — and conventions have to be decided before the second source, not discovered around the sixth.

Four conventions that make the ninth source cheap

1. One landing shape

```
raw/<source>/<table>/dt=YYYY-MM-DD/part-NNNN.parquet
```

Every source lands under the same prefix pattern, partitioned the same way. **No exceptions**, including for the source that "is a bit different" — that source is the reason the convention erodes.

The benefit is not tidiness. It is that every downstream tool, alarm, lifecycle rule and access grant can be written once against a shape rather than eight times against eight shapes.

2. One contract per source

Schema, natural key, load mode and owner declared **once**, in configuration, never buried in code.

A contract in config can be diffed in a pull request and read by someone who does not write Python. The same information spread across eight job files cannot be reviewed at all.

3. Isolation

One bad feed must not stall the other seven. That means separate runs, separate state and separate alarms per source.

The anti-pattern is a single orchestration that fails as a unit: Epsilon's API is slow, so the whole night's load is marked failed, and nobody can tell which seven sources actually succeeded.

4. Assume late and duplicate rows

Rows **will** arrive late, and they **will** arrive twice. Both are normal operating conditions, not incidents.

An idempotent `MERGE` on a key makes both harmless (Lesson 11). Without it, every late file is a manual decision and every duplicate delivery is an outage.

Decide these early — they are painful to change

DECISION	WHY IT IS PAINFUL LATER
Partition key and its format	changing it means rewriting every prefix and every downstream reference
Timezone	pick one, write it down; mixed timezones produce off-by-one-day bugs that survive for years
What "yesterday" means per source	a source in another timezone with a nightly cut has a different "yesterday" than your scheduler does

That last one causes more quiet wrongness than any other item on this list. Write it down per source.

The acceptance test

> **If adding the ninth source needs new code, the eighth was onboarded wrong.**

L24 · Eight Sources, One Platform

Check this at source two, not source eight. Onboard the second source and count the lines of new Python you had to write. If it is not close to zero, the seam is in the wrong place — fix it now, while there are two.

In practice

Eight Apparel Group sources reduce to **three connectors**:

- Three Oracle databases → one Oracle connector
- Two cursor-paged SaaS APIs → one API connector
- Two footfall file feeds → one file-drop connector

The variety lives in the specs. That is exactly why the engine is worth building in week one rather than week nine — which is where Module 2 begins.

Checklist

- ☐ Every source lands under the same prefix shape
- ☐ Every source has a contract in configuration, not in code
- ☐ One source failing cannot stall the others
- ☐ Loads are idempotent, and I have tested it
- ☐ Partition key, timezone and "yesterday" are written down
- ☐ I verified at source two that source three needs no new code

You've got it when you can…

…be handed a ninth source and estimate the work as "one spec file" — and defend that estimate by pointing at the four conventions rather than by hoping.

MODULE 0 · LESSON 25

Making It Run In Order

Four AWS options for the same job: run these things, in this order, retry what fails, and tell someone when it does.

1 · WHAT IT IS

Orchestration is where a pipeline becomes something you can operate

Whichever tool you pick must answer four questions on demand: what ran, when it ran, how long it took, and whether it succeeded.

2 · HOW IT WORKS

FOUR TOOLS, DIFFERENT CENTRES OF GRAVITY

STEP FUNCTIONS — `serverless · JSON`
A state machine with native retries, parallelism and error paths. Best when it is all AWS services.

MWAA · AIRFLOW — `Python DAGs`
Rich scheduling and a huge ecosystem — at the price of operating an Airflow environment.

GLUE WORKFLOWS — `triggers · workflows`
Built into Glue. Simple, and entirely sufficient while every step really is a Glue job.

EVENTBRIDGE — `schedules · event rules`
The clock and the tripwire. Usually sits in front of one of the other three, not instead of it.

```
what ran  ·  when  ·  how long  ·  did it succeed  —  answer all four, or you are blind
```

3 · PICK BY WHAT YOU RUN

All AWS services → Step Functions
Mixed and complex → MWAA
Only Glue jobs → Glue workflows

NEVER
A cron job on a server nobody maintains

4 · IN PRACTICE

EventBridge starts the day's run.
Step Functions sequences the phases.
Run state is queryable, not just in logs.

Module 2 builds this properly

IN PLAIN ENGLISH

The kitchen needs more than good cooks. Someone has to call the order, know what is still on, and notice when a dish never came back.

MODULE 0 · LESSON 25

L25 · Making It Run In Order

Slide: `_render/L25-orchestration.html`

What it is

Orchestration is where a pipeline becomes something you can **operate**.

Whichever tool you pick has to answer four questions on demand:

> **What ran · when it ran · how long it took · whether it succeeded.**

If the answer to any of those lives only in a log file that someone has to grep, you do not have orchestration. You have a scheduler.

Four tools, different centres of gravity

AWS Step Functions

A state machine defined as JSON, with **native retries, parallelism and error paths**. Best when everything you are sequencing is an AWS service — Glue jobs, Lambda, the Redshift Data API.

Serverless, so there is nothing to patch. The trade is that complex logic in JSON is harder to read than the equivalent Python.

Amazon MWAA (Managed Airflow)

Python DAGs, rich scheduling, backfills, and an enormous ecosystem of operators. The right choice when the pipeline is complex, mixed, or already Airflow-shaped.

The trade is that you are operating an Airflow environment — it has a cost floor, a version to upgrade and a scheduler that can itself fall over.

Glue workflows and triggers

Built into Glue. Simple, and entirely sufficient **while every step really is a Glue job**. The day you need to call something that is not Glue, you will outgrow it — so notice when that day is coming.

Amazon EventBridge

Schedules and event rules: the **clock and the tripwire**. It usually sits *in front of* one of the other three rather than instead of them — a cron expression that starts a Step Functions execution, or an S3 event that triggers a load.

Picking

WHAT YOU ARE RUNNING	TOOL
All AWS services	**Step Functions**
Mixed and complex, or existing Airflow skills	**MWAA**
Only Glue jobs, for now	**Glue workflows**
The trigger itself	**EventBridge**, in front of the above

Never: a cron job on a server nobody maintains. It works perfectly until the person who set it up leaves, and then it is unowned infrastructure with production dependencies.

What good looks like

Beyond running things in order, an orchestration layer should give you:

L25 · Making It Run In Order

- **Retries with backoff**, distinguishing transient failures from real ones
- **Backfill** — re-run a date range without hand-editing anything
- **Dependency awareness** — the gold build does not start because it is 6am; it starts because silver finished
- **Queryable run state** — "did XStore load last night?" answered with a query, not by opening logs

That last one is the difference between a pipeline you can operate and one you can only watch.

In practice

- **EventBridge** starts the day's run on a schedule.
- **Step Functions** sequences the phases and handles retries.
- **Run state is queryable**, not just present in logs — so an operations dashboard can show what ran without scraping anything.

Module 2 builds this properly, including the control plane that holds run state, watermarks and lineage.

Checklist

- ☐ I can name the four tools and what each is best at
- ☐ I know EventBridge usually fronts the others rather than replacing them
- ☐ I can state the four questions orchestration must answer
- ☐ I know why dependency-driven beats time-driven
- ☐ I would object to a cron job on an unmanaged server
- ☐ I know run state should be queryable, not just logged

You've got it when you can…

…be asked "did last night's load work?" and answer from a query in ten seconds — rather than from CloudWatch Logs in ten minutes.

MODULE 0 · LESSON 26

S3 Is Not A Filesystem

It is a key-value store of objects. Almost every data-lake mistake starts with forgetting that the folders are not real.

1 · WHAT IT IS

There are no directories — only very long keys with slashes in them

`raw/xstore/sales_line/dt=2026-08-11/part-0000.parquet ← one flat key, not four folders`

2 · HOW IT WORKS

FOUR CONSEQUENCES OF THAT ONE FACT

OBJECTS ARE WHOLE — `replace, never edit`

You cannot change one row in an object. You rewrite the object. That is why Lesson 28 exists.

PARTITIONS ARE PREFIXES — `dt=2026-08-11`

A convention the engine understands. Filter on the partition and it skips whole prefixes unread.

FILE SIZE IS A DESIGN CHOICE — `aim 128 MB – 1 GB`

Thousands of tiny files is the classic lake killer: all overhead, no throughput.

LISTING IS EXPENSIVE — `use the catalog`

Asking S3 “what is in here?” costs time and money. The catalog exists so you rarely have to.

`partitioning is the cheapest performance work you will do — and the hardest to change`

3 · RULES OF THUMB

Partition by what you filter on — date

Compact small files on a schedule

Write new, then swap — never in place

NEVER

Partition on something high-cardinality

4 · IN PRACTICE

`raw/<source>/<table>/dt=YYYY-MM-DD/`

One prefix shape for every source.

Bucket policy denies unencrypted writes.

the prefix is the contract

IN PLAIN ENGLISH

Not a filing cabinet with drawers — one enormous room where every box has a very long label. The label is the only organisation there is.

L26 · S3 Is Not A Filesystem

Slide: `_render/L26-s3-as-a-platform.html`

What it is

S3 is a **key-value store of objects**. There are no directories — only very long keys that happen to contain slashes.

```
raw/xstore/sales_line/dt=2026-08-11/part-0000.parquet
```

That is **one flat key**, not four folders. The console draws a folder tree because humans find it comforting; the underlying store has no such concept.

Almost every data-lake mistake starts with forgetting this.

Four consequences of that one fact

1. Objects are whole

You cannot change one row inside an object. You replace the object. This is precisely why table formats exist (Lesson 28) — Iceberg gives you row-level updates by adding metadata *around* immutable files, not by editing them.

2. Partitions are prefixes

`dt=2026-08-11` is a **convention** that query engines understand. When you filter on the partition column, the engine can skip entire prefixes without reading a single byte of them.

This is the cheapest performance work you will ever do — and the hardest to change afterwards, because changing it means rewriting every key and every downstream reference.

3. File size is a design choice

Thousands of tiny files is the classic lake killer: all request overhead, no throughput. Each file carries a fixed cost to open, and a query over 50,000 small files spends its life on round trips rather than on reading data.

Aim for 128 MB to 1 GB per file. If your producer naturally emits small files, run a compaction step. (S3 Tables handles this for you, which is one of the main reasons to use it.)

4. Listing is expensive

Asking S3 "what is in this prefix?" costs time and money, and gets slower as the prefix grows. The catalog exists so that engines rarely have to ask — they look up the partitions they need in metadata instead.

This is also why **partition projection** (Lesson 29) is worth knowing: it lets the engine compute partition locations from a pattern rather than listing or crawling them.

Rules of thumb

- **Partition by what you filter on** — for retail, that is almost always date
- **Compact small files** on a schedule, or let S3 Tables do it
- **Write new, then swap** — never modify in place
- **Never partition on something high-cardinality** — partitioning by customer ID creates millions of tiny prefixes and is worse than not partitioning at all

In practice

```
raw/<source>/<table>/dt=YYYY-MM-DD/
```

L26 · S3 Is Not A Filesystem

- One prefix shape for every source, with no exceptions.
- Bucket policy **denies unencrypted writes**, so encryption is not something anyone has to remember.
- **The prefix is the contract.** Downstream tooling, lifecycle rules and access grants are all written against that shape.

Checklist

- ☐ I can explain why S3 has no folders and why it matters
- ☐ I know partitions are a prefix convention, not a storage feature
- ☐ I know the target file-size range and why
- ☐ I know why listing is expensive and what avoids it
- ☐ I would never partition on a high-cardinality column
- ☐ I know our prefix convention and treat it as a contract

You've got it when you can…

…be shown a slow Athena query, look at the S3 prefix layout and the file sizes, and predict whether the problem is partitioning or small files — before running an explain.

MODULE 0 · LESSON 27

Hot Data, Cold Data

Storage classes look like a discount menu. They are really a bet about how often you will read the data again.

1 · WHAT IT IS

Cheaper to store always means more expensive, or slower, to read

There is no free tier change. Every step down the ladder moves cost from the monthly bill onto whoever runs the next query.

2 · THE LADDER

CLASS	RETRIEVAL	USE IT FOR
S3 STANDARD	instant · free	everything you query this quarter
STANDARD-IA	instant · charged	read a few times a year
GLACIER INSTANT RETRIEVAL	instant · charged	archive you must still query
GLACIER FLEXIBLE	minutes to hours	true archive, occasional restore
DEEP ARCHIVE	up to 12 hours	regulatory retention only

THE MISTAKE EVERYONE MAKES

Archiving data you still query is a cost increase, not a saving. Retrieval is billed too.

3 · THE FINE PRINT

Retrieval is charged separately, per GB

Minimum storage durations apply

Restores are not instant below IR

NEVER

Archive a partition a dashboard still reads

4 · A REAL RETENTION DESIGN

0–90 days → Standard

90 days → Standard-IA

1 year → Glacier Instant Retrieval

7 years → expire

lifecycle rules, not a person

IN PLAIN ENGLISH

Off-site storage is cheap right up until you need the box back. Cheap to keep, expensive to fetch — so only send what you will not fetch.

L27 · Hot Data, Cold Data

Slide: `_render/L27-hot-and-cold.html`

What it is

S3 storage classes look like a discount menu. They are really **a bet about how often you will read the data again**.

There is no free tier change. Every step down the ladder moves cost from the monthly storage bill onto whoever runs the next query.

The ladder

CLASS	RETRIEVAL	USE IT FOR
S3 Standard	instant, free	everything you query this quarter
Standard-IA	instant, charged per GB	read a few times a year
Glacier Instant Retrieval	instant, charged	archive you must still be able to query
Glacier Flexible Retrieval	minutes to hours	true archive, occasional restore
Deep Archive	up to ~12 hours	regulatory retention only

S3 Intelligent-Tiering sits alongside these: it moves objects between tiers based on observed access, for a small monitoring fee per object. It is a reasonable default when access patterns are genuinely unknown — and a poor one when you already know them, because you are paying for a decision you could have made yourself.

The mistake everyone makes

> **Archiving data you still query is a cost *increase*, not a saving.**

Retrieval is billed. A partition moved to Glacier Instant Retrieval to "save money", which a dashboard then scans every morning, now costs more than it did in Standard — and nobody notices, because the saving appears on the storage line and the cost appears on the retrieval line.

Before any lifecycle rule, ask: **what still reads this?** If the answer is "a report", the rule is wrong.

The fine print that catches people

- **Retrieval is charged separately**, per GB, and it is not small at the colder tiers
- **Minimum storage durations apply** — deleting an object early still bills you for the minimum period
- **Restores below Instant Retrieval are not instant.** Deep Archive can take most of a day, which means it cannot serve an urgent audit request without notice
- **Per-object monitoring fees** on Intelligent-Tiering make it a poor fit for very many small objects

A real retention design

AGE	CLASS	WHY
0–90 days	**Standard**	actively queried; reprocessing happens here
90 days	**Standard-IA**	occasionally re-read for investigation
1 year	**Glacier Instant Retrieval**	still queryable for audit, rarely touched
7 years	**expire**	retention period ends

L27 · Hot Data, Cold Data

The important property: this is expressed as **lifecycle rules**, not as a person remembering. A retention policy that depends on someone running a script every quarter is not a policy.

The connection to Lesson 26

Lifecycle rules act on **prefixes**. Which means your partitioning scheme decides what you can tier. Partitioned by date, "everything older than a year" is one rule. Not partitioned by date, it is not expressible at all.

Another reason the prefix convention is a contract, not a preference.

Checklist

- ☐ I can name the five classes in order and their retrieval behaviour
- ☐ I know retrieval is billed separately
- ☐ I know minimum storage durations exist
- ☐ I ask "what still reads this?" before writing a lifecycle rule
- ☐ I express retention as lifecycle rules, not as a manual process
- ☐ I understand why date partitioning is what makes tiering possible

You've got it when you can…

…be handed a cost report showing a rising S3 retrieval line, trace it to a lifecycle rule someone added to save money, and explain why it did the opposite.

MODULE 0 · LESSON 28

Two Different Things Called “Format”

Parquet is a **file** format. Iceberg is a **table** format. They stack on top of each other, and people conflate them constantly.

1 · ONE SITS ON TOP OF THE OTHER

FILE FORMATS

what one file looks like

CSV · JSON

Readable, untyped, uncompressed. Fine for landing, never for anything you query repeatedly.

PARQUET

Columnar, typed, compressed. The default for the lake: it can cut bytes scanned by 10–100×.

ORC

Also columnar, common in Hive estates. Parquet is the AWS default; both work.

AVRO

Row-based with an embedded schema. Good for streaming records, poor for analytical scans.

```
columnar + compressed = the biggest cost lever
```

TABLE FORMATS

what a set of files means

WHAT THEY ADD

ACID commits · schema evolution · row-level UPDATE and DELETE · time travel · safe concurrency

APACHE ICEBERG

The AWS default. S3 Tables is managed Iceberg. v2 adds delete files for row-level changes.

APACHE HUDI

Copy-on-Write and Merge-on-Read variants. Strong on high-frequency streaming upserts.

DELTA LAKE

Databricks’ format. Redshift Spectrum reads it through symlink manifest tables.

```
metadata about which files are in the table now
```

IN PLAIN ENGLISH

Parquet is how one page is printed. Iceberg is the index at the front saying which pages are currently part of the book.

MODULE 0 · LESSON 28

L28 · Two Different Things Called "Format"

Slide: `_render/L28-file-and-table-formats.html`

What it is

Parquet is a file format. Iceberg is a table format. They stack on top of each other, and people conflate them constantly — including in job descriptions and architecture documents.

- A **file format** decides what **one file** looks like inside.
- A **table format** decides what **a set of files** collectively means.

You choose both, and they are independent decisions.

File formats — what one file looks like

CSV · JSON

Readable, untyped, uncompressed. Fine for landing raw data exactly as received. Never for anything you query repeatedly — every query re-parses text and reads every column.

Parquet

Columnar, typed, compressed. The default for the lake. Because it stores columns separately and keeps per-column statistics, a query touching a few columns with a selective filter can cut bytes scanned by **10–100×**.

Given that Athena is billed per byte scanned, that is not a performance number — it is a bill.

ORC

Also columnar, with similar properties. Common in Hive estates. Parquet is the AWS default; either works, and mixing them within one table does not.

Avro

Row-based with an embedded schema. Good for streaming records and message payloads, where you write whole rows and read whole rows. Poor for analytical scans, for the same reason row storage is always poor for them.

Table formats — what a set of files means

What they add

- **ACID commits** — a write becomes visible in its entirety or not at all
- **Schema evolution** — add, rename, drop columns without rewriting data
- **Row-level UPDATE and DELETE** — on storage that cannot edit files
- **Time travel** — query the table as of an earlier snapshot
- **Safe concurrency** — two writers no longer corrupt each other

Apache Iceberg

The AWS default. **S3 Tables is managed Iceberg**, with AWS running compaction and snapshot maintenance.

v2 introduced *delete files*: rather than rewriting a whole data file to remove a row, Iceberg writes a small file recording the deletion, and readers apply it. Fast writes, slightly slower reads, and compaction reconciles the two later. That trade is worth understanding because it explains why maintenance is not optional.

Apache Hudi

Two variants: **Copy-on-Write** (rewrites files on update — fast reads, slower writes) and **Merge-on-Read** (writes deltas — fast writes, slower reads). Strong on high-frequency streaming upserts.

L28 · Two Different Things Called "Format"

Delta Lake

Databricks' format. **Redshift Spectrum reads it through symlink manifest tables** — a generated manifest that lists the current files, which Spectrum then reads as an external table.

How to talk about this correctly

> "Iceberg tables stored as Parquet files on S3."

That sentence has all three layers in the right order: table format, file format, storage. If someone says "Parquet versus Iceberg" as if it were a choice between two things, they have the layers confused — and it is worth gently untangling, because the confusion leads to real design errors.

Checklist

- ☐ I can state the difference between a file format and a table format
- ☐ I know why columnar plus compression is the biggest cost lever in a lake
- ☐ I can name the five things a table format adds
- ☐ I know what Iceberg v2 delete files are and why compaction matters
- ☐ I know Delta is read via symlink manifests in Spectrum
- ☐ I would correct "Parquet versus Iceberg" in a design document

You've got it when you can…

…read an architecture document that says "we'll use Parquet or Iceberg" and explain, without condescension, that those are two different layers and you almost certainly want both.

MODULE 0 · LESSON 29

The Catalog Is Now Federated

It is no longer one flat list of databases. It is a three-level hierarchy that can mount other systems in place, without copying them.

1 · WHAT IT IS

Three levels — and the top one can be somebody else's system

`catalog . database . table ← nested catalogs mirror the shape of the source`

2 · HOW IT WORKS

TWO KINDS OF CATALOG, ONE GOVERNANCE PLANE

MANAGED CATALOG — S3 or RMS

A catalog you create. The data is managed for you, in S3 or in Redshift Managed Storage.

FEDERATED CATALOG — mounts, does not copy

Points at a system that already exists. The tables appear; the bytes never move.

ONE GOVERNANCE PLANE — AWS Lake Formation

Grant at catalog, database, table or column — and every engine reading the catalog enforces it.

ANY ICEBERG ENGINE — Athena · Redshift · Spark · EMR

Query in place from whichever engine suits the job. One copy of the data, many ways in.

```
this is the SageMaker Lakehouse architecture: S3 lakes + Redshift, one catalog
```

3 · WHAT YOU CAN MOUNT

Amazon Redshift · DynamoDB · DocumentDB
Oracle · SQL Server · MySQL · PostgreSQL
Aurora MySQL · Aurora PostgreSQL
BigQuery · Snowflake · Azure SQL

`S3 Tables → s3tablescatalog`

4 · GOTCHAS

Lakehouse identifiers must be lowercase.
Prefer declarative registration to crawlers.
Partition projection beats crawling.

decide naming before you build

IN PLAIN ENGLISH

A library card catalogue that also lists the books in the library across the road — and the librarian there still checks your card.

L29 · The Catalog Is Now Federated

Slide: `_render/L29-federated-catalog.html`

What it is

The Glue Data Catalog is **no longer one flat list of databases**. If your mental model is "a place that remembers table schemas so Athena can query S3", it is several years behind.

It is now a **three-level hierarchy**:

```
catalog  .  database  .  table
```

with **nested catalogs** allowed, so the hierarchy can mirror the shape of the system underneath.

Two kinds of catalog

Managed catalog

A catalog you create. The data is managed for you, stored in either **Amazon S3** or **Redshift Managed Storage (RMS)**.

Federated catalog

Mounts an existing data source in place, without copying anything. The tables appear in the catalog and become queryable; the bytes never move and the source system remains the system of record.

What you can mount

SOURCE	
Amazon Redshift	Amazon DynamoDB
Amazon DocumentDB	MySQL
PostgreSQL	SQL Server
Oracle	Aurora MySQL
Aurora PostgreSQL	Google BigQuery
Snowflake	Microsoft Azure SQL

Plus **Amazon S3 Tables**, which appears as `s3tablescatalog`.

Note **Oracle** on that list. For Apparel Group that means Oracle reference and lookup data could be *mounted* rather than ingested — a genuine third option alongside a Glue pipeline and zero-ETL (Lesson 21), and one worth considering per table rather than per source.

One governance plane

AWS Lake Formation grants at **catalog → database → table → column** level, and — this is the part that makes it worth the complexity — the grant is **enforced by every engine that reads through the catalog**. Athena, Spark, EMR and Redshift Spectrum all honour it.

Define the permission once; it applies everywhere. Compare that to the alternative, which is the same rule re-implemented in four places and drifting apart.

Any Iceberg-compatible engine

Because the catalog and the table format are open, you query in place from whichever engine suits the job — Athena for exploration, Redshift for reporting, Spark for heavy transformation — against **one copy** of the data.

This is the **SageMaker Lakehouse architecture**: S3 data lakes and Redshift warehouses unified into one catalog.

L29 · The Catalog Is Now Federated

> ⚠️ Product naming in this area has moved recently (SageMaker Lakehouse, SageMaker Unified Studio, DataZone). Check the current names against AWS documentation before quoting them in a client deck.

Gotchas

- **Lowercase identifiers.** The lakehouse architecture currently supports lowercase table, column and database names. Plan naming conventions accordingly — retrofitting this is painful.
- **Prefer declarative registration to crawlers.** A crawler infers schema by sampling, which means the schema can change because the data changed, silently. Mature platforms register tables explicitly so that a schema change is a pull request.
- **Partition projection beats crawling.** If partitions follow a predictable pattern, configure projection so the engine computes locations rather than listing or crawling them. Faster, cheaper, and it cannot fall behind.

Checklist

- ☐ I can state the three levels of the hierarchy
- ☐ I know the difference between a managed and a federated catalog
- ☐ I can name at least six sources that can be mounted federated
- ☐ I know Lake Formation grants are enforced across engines
- ☐ I know the lowercase-identifier constraint
- ☐ I know why crawlers are avoided on mature platforms
- ☐ I would re-verify the product naming before quoting it externally

You've got it when you can…

…be asked "how do we get the Oracle reference tables into the lake?" and offer three real options — ingest, replicate, or mount as a federated catalog — with the trade-offs of each.

MODULE 0 · LESSON 30

Athena Also Writes

Serverless SQL over the catalog, priced per terabyte scanned. “Athena is read-only” is the most common wrong belief in this whole stack.

1 · WHAT IT IS

No cluster to start, no capacity to size — and a real ETL tool, not just a query box

You pay per byte scanned. That single pricing fact is why partitioning and Parquet are cost controls, not just performance tuning.

2 · HOW IT WORKS

IT READS EVERYTHING IN THE CATALOG — AND WRITES TOO

READS — the shared catalog

S3 files, Iceberg tables, and through connectors Redshift, DynamoDB, Snowflake and more.

CTAS — CREATE TABLE AS SELECT

Writes a whole new table and its files. This is how people build lake tables without Spark.

INSERT INTO · ICEBERG DML — UPDATE · DELETE · MERGE

Appends to an existing table. On Iceberg you also get row-level DML and time travel.

THE WRITE LIMITS — know these before you design

100 partitions per CTAS or INSERT statement. Not supported for bucketed tables.

```
priced per TB scanned — partitioning and Parquet are how you control the bill
```

3 · TWO CONNECTOR TYPES

ATHENA DATA CATALOG CONNECTOR

A Lambda in your account. Cannot register as a federated catalog — so no Lake Formation control.

GLUE DATA CATALOG CONNECTOR

Uses a Glue connection, registers as a federated catalog, and does support fine-grained control.

4 · IN PRACTICE

Athena is how you inspect the lake by hand.
Workgroups cap bytes scanned per query.
Same catalog as Glue, Redshift and EMR.

prefer the Glue-connection type

IN PLAIN ENGLISH

A meter on the tap, not a monthly rental. You pay for what you draw — so a badly shaped question is expensive, not just slow.

L30 · Athena Also Writes

Slide: `_render/L30-athena-read-and-write.html`

What it is

Athena is serverless SQL over the Glue Data Catalog. No cluster to start, no capacity to size, and you pay **per byte scanned**.

That pricing model is the single most important fact about it: **partitioning and Parquet are cost controls, not just performance tuning**. A badly shaped query is expensive, not merely slow, and that changes how people write SQL.

"Athena is read-only" is the most common wrong belief in this whole stack. It writes.

What it reads

Anything in the catalog: S3 files, Iceberg tables, and — through connectors — Redshift, DynamoDB, Snowflake and others.

What it writes

CTAS — `CREATE TABLE AS SELECT`

```
CREATE TABLE silver.store_daily
WITH (format = 'PARQUET', partitioned_by = ARRAY['dt'])
AS SELECT ... FROM bronze.sales_line ...;
```

Writes a whole new table **and its files**. This is how people build lake tables without ever standing up Spark — a genuinely underused capability on smaller platforms.

`INSERT INTO`

Appends to an existing table.

Iceberg DML

On Iceberg tables you also get `UPDATE`, `DELETE`, `MERGE` **and time travel** — full row-level SQL against data in the lake.

The write limits — know these before you design

- **100 partitions per CTAS or INSERT statement.** A backfill across two years of daily partitions cannot be one statement; it has to be chunked.
- **Not supported for bucketed tables.**

The 100-partition limit is the one that bites. It is not a reason to avoid Athena writes — it is a reason to write the loop.

The two connector types — and the difference matters

Athena federated query comes in two flavours, and they are not equivalent:

TYPE	HOW IT WORKS	GOVERNANCE
Athena data catalog federated connector	a **Lambda function in your account**	cannot register as a Glue federated catalog → **no Lake Formation fine-grained control**
Glue Data Catalog federated connector	uses a **Glue connection**	registers as a federated catalog → **full Lake Formation control**

Prefer the Glue-connection type. If governance matters at all — and on a platform holding PII it does — the Lambda-based type puts your access control outside the system that is supposed to own it.

Cost governance

Workgroups let you cap bytes scanned per query and per workgroup, route results to a specific S3 location, and separate teams for cost attribution.

L30 · Athena Also Writes

Set a per-query limit. It converts "someone ran an accidental cross join over the whole lake" from an invoice into an error message.

In practice

- Athena is how you **inspect the lake by hand** — the first tool you reach for when investigating a number.
- Workgroups cap bytes scanned per query.
- It reads the **same catalog** as Glue, Redshift and EMR, so what you see in Athena is what the pipeline sees.

Checklist

- ☐ I know Athena writes via CTAS and INSERT INTO
- ☐ I know Iceberg tables get full DML and time travel
- ☐ I know the 100-partition-per-statement limit
- ☐ I know there are two connector types and which one supports Lake Formation
- ☐ I have a per-query scan limit set on my workgroup
- ☐ I understand why per-byte pricing changes how SQL gets written

You've got it when you can…

…be told "we need Spark to build that table" for a modest transformation, and correctly identify that a CTAS would do it — including whether the partition count fits in one statement.

MODULE 0 · LESSON 31

One Query, Three Systems

An external schema is a pointer. Once you have four kinds of pointer, one SQL statement can span the warehouse, the lake and production.

1 · WHAT IT IS

The table appears in your database. The bytes stay where they were.

```
SELECT ... FROM gold.fct_sales f JOIN lake_ext.silver_store s USING(...) JOIN live_pg.customers c USING(...)
```

2 · FOUR DOORS, FOUR DESTINATIONS

ALL FOUR LOOK LIKE AN ORDINARY SCHEMA IN SQL

REDSHIFT → S3 — `Redshift Spectrum`
External schema from the Glue catalog. Reads the lake without ever loading it in.

REDSHIFT → LIVE DATABASE — `federated query`
External schema onto RDS or Aurora PostgreSQL / MySQL, with predicates pushed down.

ATHENA → ANYTHING — `federated connectors`
The same catalog, plus connectors out to Redshift, DynamoDB, Snowflake and others.

REDSHIFT → REDSHIFT — `datashares`
Another warehouse's live tables, across clusters, accounts and Regions.

```
an external schema is a pointer — the work still happens, just somewhere else
```

3 · WHO ACTUALLY PAYS

Spectrum — bytes scanned in S3
Federated — load on the production DB
Datashare — the consumer's own compute

NEVER
Assume a pointer makes the work free

4 · IN PRACTICE

An external schema exposes S3 Tables data.
Spectrum's read support is broad.
Its write support is narrow and format-specific.

verify before promising a write path

IN PLAIN ENGLISH

Four doors, all painted the same. The SQL looks identical through each one; the bill and the blast radius do not.

L31 · One Query, Three Systems

Slide: `_render/L31-external-schemas.html`

What it is

An **external schema is a pointer**. The table appears in your database and the bytes stay where they were.

Once you have four kinds of pointer, a single SQL statement can span the warehouse, the lake and a production database:

```
SELECT  s.region, d.month, SUM(f.net_amount), COUNT(DISTINCT c.customer_id)
FROM        gold.fct_sales_line   f            -- Redshift, loaded
JOIN        lake_ext.dim_store    s  USING (store_sk)   -- S3 / Iceberg, pointed at
JOIN        gold.dim_date         d  USING (date_sk)
LEFT JOIN   live_pg.customers     c  USING (customer_id) -- live Aurora, federated
GROUP BY 1, 2;
```

Three storage systems, one statement, no ETL between them. It is genuinely powerful and genuinely easy to misuse.

Four doors

DOOR	REACHES	MECHANISM
Redshift → S3	the lake, via the catalog	Redshift Spectrum
Redshift → live database	RDS / Aurora PostgreSQL or MySQL	federated query, with predicate pushdown
Athena → anything	Redshift, DynamoDB, Snowflake and more	federated connectors (Lesson 30)
Redshift → Redshift	another warehouse, live	datashares, cross-account and cross-Region

All four look like an ordinary schema in SQL. That is the point, and also the risk — nothing in the query text tells you which door you just opened.

Who actually pays

DOOR	THE BILL LANDS ON
Spectrum	bytes scanned in S3, per query
Federated query	the production database, per query
Datashare	the consumer's own compute
Athena connector	Athena scan cost, plus the remote system

Never assume a pointer makes the work free. The work still happens; it happens somewhere else, and often on someone else's budget or someone else's production system.

This is worth making explicit in design reviews, because the cost of a zero-copy design is invisible in the design and highly visible in month three.

Spectrum's write support — flagged honestly

Spectrum's **read** support is broad and well documented: Parquet, ORC, CSV, JSON, Avro, plus Iceberg, Hudi Copy-on-Write and Delta Lake via symlink manifests.

Its **write** support is narrow and format-dependent. Before you promise anyone a write path through Spectrum, verify it for the specific format and table type you have — do not assume symmetry with reads.

The general pattern to prefer: **write with the engine that owns the table** (Glue/Spark or Athena for lake tables, Redshift for warehouse tables), and use external schemas for reading.

L31 · One Query, Three Systems

In practice

- An external schema exposes S3 Tables data inside Redshift, so Redshift reads Iceberg it never loaded.
- **Gold is loaded, silver is pointed at** — which is Lesson 18's rule applied.
- Every external schema has a documented purpose and an expected query frequency, so that "someone pointed a dashboard at production" is caught in review rather than in an incident.

Checklist

- ☐ I can name the four doors and what each reaches
- ☐ I can say who pays for each
- ☐ I can write an external schema definition for Spectrum and for federated query
- ☐ I know Spectrum's read formats, including the three table formats
- ☐ I would verify Spectrum write support rather than assume it
- ☐ I document what each external schema is for

You've got it when you can…

…write a query spanning the warehouse, the lake and a live database — and then explain to the person paying the bill exactly which part of it costs what, and what would happen if it ran hourly.

MODULE 0 · LESSON 32

The Whole Thing, On One Slide

Eight sources on the left, a dashboard on the right — and the name of an AWS service on every single arrow between them.

1 · THE REFERENCE ARCHITECTURE

SOURCES

Oracle Retail (RMS)
Oracle SIM
Oracle XStore
Epsilon · MoEngage
Magento
Vemco · Irisys files

on-prem, SaaS and files — five different classes

`VPN / Direct Connect`

INGEST

AWS Glue (Spark)
AWS DMS (CDC)
Zero-ETL integrations
Transfer Family
AppFlow
Firehose (if streaming)

one connector per class, one spec per table

`Lesson 20`

LAKE — S3 + ICEBERG

raw/ — immutable landing, as received
bronze/ — Iceberg, typed, deduplicated
silver/ — Iceberg, conformed, joined
Amazon S3 Tables (managed Iceberg)
Lifecycle: hot → IA → Glacier IR
MERGE on a key — never blind append

one copy of the data; every engine reads it through the catalog

`Lessons 26–29`

WAREHOUSE

Redshift Serverless
gold/ — facts and dimensions
Built by dbt, MERGE-loaded
Materialized views for BI
Spectrum reads silver in place
RLS / CLS on sensitive columns

load what you join hard; point at the rest

`Lessons 10–18`

CONSUME

Power BI (reporting)
Amazon QuickSight
Athena (ad-hoc SQL)
SageMaker (ML)
Ops & monitoring console
UNLOAD back to S3

BI reads views, never the underlying fact tables

`Lesson 15`

UNDERNEATH ALL FIVE COLUMNS, ALL OF THE TIME

Glue Data Catalog (federated) · Lake Formation (fine-grained access) · Step Functions + EventBridge (orchestration) · CloudWatch (observability) · Terraform (all of it, as code)

IN PLAIN ENGLISH

Goods in, unpacked, sorted, priced, put on the shelf, sold. The catalog is the stock system and Lake Formation is the security guard.

L32 · The Whole Thing, On One Slide

Slide: `_render/L32-reference-architecture.html`

What it is

Eight sources on the left, a dashboard on the right, and the name of an AWS service on every arrow between them.

Everything in this module appears somewhere on this slide. If a service on it is unfamiliar, the lesson number is on the diagram.

The five columns

Sources

Oracle RMS · Oracle SIM · Oracle XStore · Epsilon · MoEngage · Magento · Vemco · Irisys

On-prem, SaaS and files — **five different classes** (Lesson 19). On-prem access arrives over VPN or Direct Connect, which is procurement, not code, and belongs on the week-one critical path.

Ingest

AWS Glue (Spark) · AWS DMS (CDC) · Zero-ETL integrations · Transfer Family · AppFlow · Firehose if streaming.

One connector per class, one spec per table (Lesson 20). The variety lives in configuration.

Lake — S3 + Iceberg

- `raw/` — immutable landing, exactly as received
- `bronze/` — Iceberg, typed, deduplicated
- `silver/` — Iceberg, conformed, joined

On **Amazon S3 Tables** (managed Iceberg). Lifecycle moves cold partitions hot → IA → Glacier IR. Loads **MERGE on a key**, never blind append.

One copy of the data; every engine reads it through the catalog.

Warehouse

Redshift Serverless holds `gold/` — facts and dimensions, built by dbt and MERGE-loaded, with materialized views for BI. **Spectrum reads silver in place** rather than loading it. RLS and CLS protect sensitive columns.

Load what you join hard; point at the rest (Lesson 18).

Consume

Power BI · QuickSight · Athena for ad-hoc · SageMaker for ML · the ops console · `UNLOAD` back to S3 for other engines.

BI reads views, never the underlying fact tables (Lesson 15).

Underneath all five columns, all of the time

LAYER	SERVICE
Catalog	**Glue Data Catalog** (federated)
Access control	**Lake Formation** (catalog / database / table / column)
Orchestration	**Step Functions + EventBridge**
Observability	**CloudWatch**
Everything above, as code	**Terraform**

These are not a footnote. They are what makes the five columns a platform rather than five projects that happen to share a bucket.

L32 · The Whole Thing, On One Slide

How to read this diagram in an interview or a client meeting

Left to right is the **data path**. Bottom to top is the **control path**. Almost every question someone asks is one of:

- *"Where does X land?"* — a column
- *"Who is allowed to see it?"* — Lake Formation
- *"How do I know it ran?"* — orchestration and observability
- *"What happens when it breaks?"* — isolation (Lesson 24) plus idempotency (Lesson 11)

Being able to point at the right box is most of the job.

Checklist

- ☐ I can draw the five columns from memory
- ☐ I can name at least three services in each column
- ☐ I can name the five cross-cutting layers
- ☐ I know which layer is loaded into Redshift and which is pointed at
- ☐ I can say where governance is enforced and by what
- ☐ I can answer the four common questions by pointing at a box

You've got it when you can…

…draw this on a whiteboard from memory in five minutes, and answer "where would a new source go?" by pointing rather than by thinking.

MODULE 0 · LESSON 33

The Whole AWS Analytics Stack

Every service in this module, grouped by what it is for, each with a one-line “use this when”. This is the poster for the wall.

1 · EIGHT GROUPS — EVERY SERVICE HAS A JOB

STORAGE

Amazon S3 — the raw substrate
S3 Tables — managed Iceberg
Redshift Managed Storage
Storage classes — hot to archive

USE WHEN
always — everything ends up here

CATALOG & GOVERNANCE

Glue Data Catalog (federated)
Lake Formation — who sees what
AWS RAM — cross-account shares
SageMaker Lakehouse

USE WHEN
more than one engine or team reads

INGEST

AWS Glue · EMR — batch ETL
AWS DMS — CDC replication
Zero-ETL — no pipeline at all
AppFlow · Transfer Family · DataSync

USE WHEN
data has to move at all — Lesson 20

STREAMING

Kinesis Data Streams
Amazon MSK · Kafka
Data Firehose — managed delivery
Managed Flink · Glue streaming

USE WHEN
seconds change a decision — rarely

QUERY & COMPUTE

Amazon Redshift — the warehouse
Athena — serverless SQL, read+write
EMR · Glue Spark — big transforms
Spectrum · federated query

USE WHEN
someone asks a question

ORCHESTRATION

Step Functions — AWS-native flows
MWAA (Airflow) — Python DAGs
Glue workflows — Glue-only jobs
EventBridge — clock and tripwire

USE WHEN
more than one step has to run in order

CONSUMPTION

Power BI · Tableau — via JDBC
Amazon QuickSight
SageMaker · Redshift ML
Redshift Data API — applications

USE WHEN
a human or an app needs the answer

OPERATIONS

CloudWatch — metrics, logs, alarms
CloudTrail — who did what
Cost Explorer · budgets
Terraform — every box above, as code

USE WHEN
from day one, not after go-live

IN PLAIN ENGLISH

Nobody memorises forty services. You memorise eight jobs, and then look up which tool does the job you actually have today.

L33 · The Whole AWS Analytics Stack

Slide: `_render/L33-ecosystem-map.html`

What it is

Every service in this module, grouped by **what it is for**, each with a one-line "use this when".

Nobody memorises forty AWS services. You memorise **eight jobs**, and then look up which tool does the job you actually have today.

The eight groups

1. Storage

Amazon S3 · S3 Tables (managed Iceberg) · Redshift Managed Storage · storage classes from hot to archive. **Use when:** always. Everything ends up here.

2. Catalog & governance

Glue Data Catalog (federated) · Lake Formation · AWS RAM for cross-account shares · SageMaker Lakehouse. **Use when:** more than one engine or more than one team reads the data. Which is immediately.

3. Ingest

AWS Glue · EMR for batch ETL · AWS DMS for CDC · zero-ETL integrations · AppFlow · Transfer Family · DataSync. **Use when:** data has to move at all — see the decision table in Lesson 20.

4. Streaming

Kinesis Data Streams · Amazon MSK / Kafka · Data Firehose · Managed Flink · Glue streaming ETL. **Use when:** seconds genuinely change a decision. Rarely, in retail reporting.

5. Query & compute

Amazon Redshift · Athena (serverless, reads **and** writes) · EMR and Glue Spark for heavy transforms · Spectrum and federated query for reading in place. **Use when:** someone asks a question.

6. Orchestration

Step Functions · MWAA (Airflow) · Glue workflows · EventBridge. **Use when:** more than one step has to run in a defined order — which is from the second job onward.

7. Consumption

Power BI and Tableau via JDBC · Amazon QuickSight · SageMaker and Redshift ML · Redshift Data API for applications. **Use when:** a human or an application needs the answer.

8. Operations

CloudWatch for metrics, logs and alarms · CloudTrail for who did what · Cost Explorer and budgets · **Terraform for every box above, as code**. **Use when:** from day one, not after go-live.

How to actually use this

When you meet a new requirement, do not start from services. Start from the **job**:

1. Which of the eight groups does this need touch?
2. Within that group, what are the two or three candidates?
3. What separates them — latency, cost model, how managed, who operates it?

That sequence turns "which of forty services?" into "which of three?", every time.

L33 · The Whole AWS Analytics Stack

The one to notice

Operations is a group, not an afterthought. It is listed eighth because that is where teams put it, and it belongs first. A platform with no observability is not finished; it is undelivered with the invoice already sent.

Checklist

- ☐ I can name the eight groups
- ☐ I can name three services in each group
- ☐ I know which group each service in this module belongs to
- ☐ I start from the job, not from the service
- ☐ I have this slide printed or saved somewhere I will see it
- ☐ I treat observability as part of the build, not a follow-up

You've got it when you can…

…be given a requirement you have never seen and narrow it to two or three candidate services in under a minute — by naming the group first.

MODULE 0 · LESSON 34

The Cost And Latency Ladder

Everything in this module on two axes. Where a thing sits on the ladder tells you exactly what having it will cost you.

1 · WHAT IT IS

Freshness is bought, and it is never bought once

Every step up this ladder is a recurring cost someone pays forever. Buy the freshness a decision actually needs, and not one rung more.

2 · THE LADDER

MECHANISM	FRESHNESS	WHAT YOU PAY, FOREVER
NIGHTLY BATCH	yesterday	cheapest per row, most control
HOURLY BATCH	within the hour	same shape, 24× the compute runs
ZERO-ETL · CDC	minutes	no pipeline to run — and no transform
FEDERATED QUERY	live	the source system, on every query
STREAMING	seconds	always-on cost, even at 3am on Sunday

AND THE ONE THAT GOES SIDEWAYS

A materialized view buys read speed for storage plus a refresh — not for freshness.

3 · ASK THESE FIVE, EVERY TIME

1. Who reads this, and how often?
2. How fresh does it truly need to be?
3. Must I transform it on the way?
4. Who pays when someone queries it?
5. What happens the day it breaks?

4 · WHERE YOU GO NEXT

Module 1 — how our platform is built.
Module 2 — how to build the next one.
Every choice there is one of these.

you now know the vocabulary

IN PLAIN ENGLISH

Next-day delivery is cheap. Same-hour costs more. Live courier costs most of all — and nobody upgrades the postage on a birthday card.

L34 · The Cost And Latency Ladder

Slide: `_render/L34-cost-and-latency-ladder.html`

What it is

Everything in this module, placed on two axes: **freshness** and **cost**.

The principle underneath it:

> **Freshness is bought, and it is never bought once.**

Every step up the ladder is a **recurring** cost that someone pays forever. Buy the freshness a decision actually needs, and not one rung more.

The ladder

MECHANISM	FRESHNESS	WHAT YOU PAY, FOREVER
Nightly batch	yesterday	cheapest per row, most control
Hourly batch	within the hour	same shape, 24× the compute runs
Zero-ETL · CDC	minutes	no pipeline to run — and no transform
Federated query	live	the source system, on every query
Streaming	seconds	always-on cost, even at 3am on Sunday

And the one that goes sideways

A **materialized view** buys read speed in exchange for storage plus a refresh. It does **not** buy freshness — it is at best as fresh as its last refresh. Reaching for an MV to solve a freshness problem is a category error, and a common one.

The five questions

Ask these before adding anything to an architecture:

1. Who reads this, and how often?

Determines whether it earns storage or should be pointed at (Lesson 18). "The finance team, monthly" and "every dashboard, hourly" are different systems.

2. How fresh does it truly need to be?

Not what was requested — what the **decision** needs. "Real-time" in a requirements document usually means "not yesterday's". Find out what someone does differently at 9am than at 9pm, and if the answer is nothing, you have a batch requirement.

3. Must I transform it on the way?

If yes, managed replication and zero-ETL are out for that source (Lesson 21). This question alone eliminates half the options on most sources.

4. Who pays when someone queries it?

Pointer-based designs move the cost onto the querier, or onto someone else's production system (Lesson 31). Make that explicit while it is a design, not a surprise in month three.

5. What happens the day it breaks?

Every rung has a different failure. Batch is late. CDC falls behind. Federated query takes production with it. Streaming drops events. Pick the failure you can staff and detect.

L34 · The Cost And Latency Ladder

Where you go next

- **Module 1** — how our platform is actually built: the medallion layers, the catalog, the repo.
- **Module 2** — how to build the next one: the decisions, the setup and the reasoning, aimed at the Apparel Group build.

Every architectural choice in those modules is one of the choices on this ladder, made once and lived with. **You now know the vocabulary** — the rest is judgement, and judgement is what the next two modules are for.

Checklist

- ☐ I can place any mechanism on the ladder
- ☐ I know an MV buys read speed, not freshness
- ☐ I ask "what decision changes?" before accepting a freshness requirement
- ☐ I ask who pays for every pointer-based design
- ☐ I ask what the failure mode is before choosing a rung
- ☐ I can explain the recurring nature of freshness cost to a non-engineer

You've got it when you can…

…sit in a requirements session, hear "we need it in real time", ask the five questions, and land on the honest answer — which is very often an hourly batch that costs a fraction of what was about to be built.